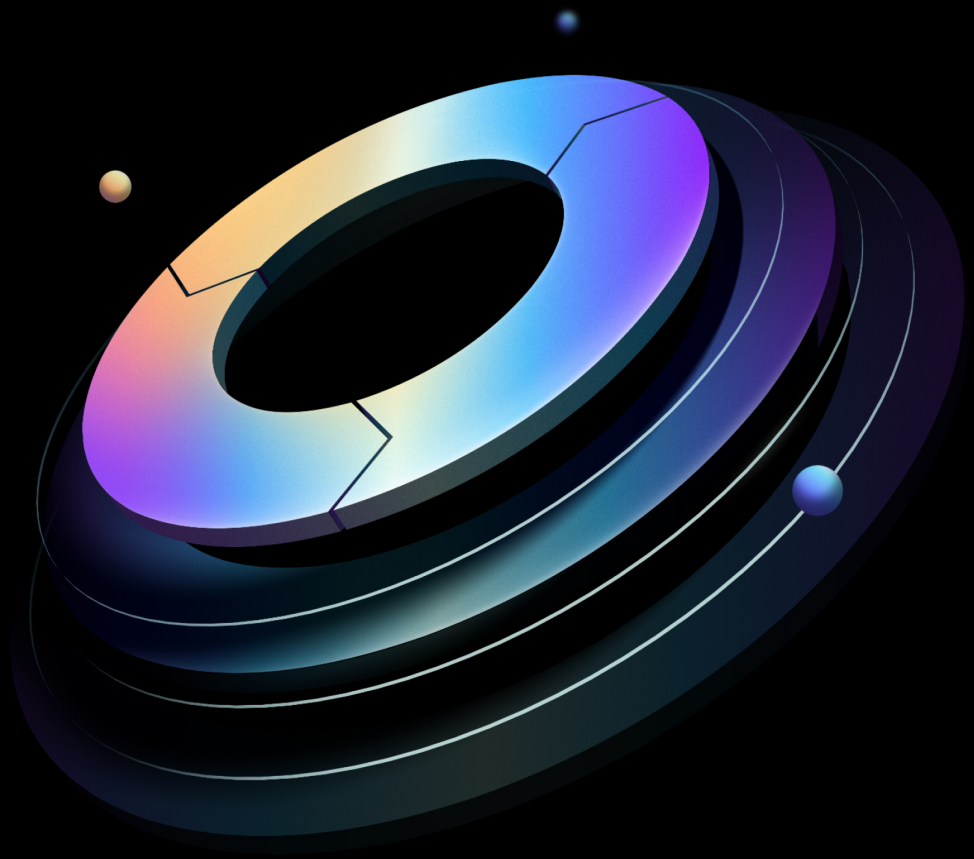
HashiCorp

Vault User Guide

Built from commit 3a6a26c83c

HashiCorp | Validated Designs

09 November 2026



HashiCorp

Contents

1 Introduction **5**
1.1 Why use HashiCorp Validated Designs? . . . 5
1.2 Audience . . . 5
1.3 Use cases covered . . . 6
1.4 Prerequisites . . . 7

2 Static secrets **7**
2.1 Consumption of secrets . . . 7
2.1.1 Secret consumption patterns . . . 7
2.1.2 Vault Agent . . . 9
2.1.3 Vault proxy . . . 13
2.1.4 The Vault Agent advantage . . . 13
2.1.5 Vault Agent best practices . . . 14
2.1.6 Configure the Vault Agent . . . 17

3 Secrets Sync **25**
3.1 Example scenario . . . 26

4 Dynamic secrets **29**
4.1 Application considerations for dynamic credentials . . . 29

5 PKI certificates **30**
5.1 A pattern for automatic certificate renewal . . . 30
5.2 Producers and consumers of certificates . . . 31
5.3 Vault Agent for VM-based workloads . . . 31
5.3.1 Introduction to Vault Agent . . . 31
5.3.2 PKI consumption workflow . . . 32
5.3.3 Implementation guide . . . 34

6 Key lifecycle management **43**
6.1 Overview . . . 43
6.1.1 Bring Your Own Key (BYOK) . . . 43

7 Transit and Transform secrets engines 44
7.1 Overview . . . 44
7.1.1 Common use cases mapped to each encryption engine . . . 45
7.2 Transit engine . . . 45
7.2.1 Design considerations . . . 46
7.2.2 Implementation . . . 47
7.2.3 Use cases covered . . . 47
7.2.4 Best practices . . . 49
7.3 Transform engine . . . 49
7.3.1 Implementation . . . 50
7.3.2 Use cases covered . . . 50
7.3.3 Format preserving encryption . . . 51
7.3.4 Tokenization . . . 52
7.3.5 Data Masking . . . 54
7.3.6 Managing roles and access . . . 55
7.3.7 Implementation best practices . . . 56
7.3.8 Performance characteristics . . . 56

8 Transit and Transform examples 57
8.1 Database column-level encryption with the Transit engine . . . 57
8.2 Securing sensitive healthcare data with the Transform engine . . . 58
8.3 Implementing GDPR compliance using Transform tokenization . . . 61
8.3.1 Using curl with Vault's API . . . 62
8.3.2 Example Outputs . . . 63
8.3.3 Benefits . . . 63

9 RHEL local account password rotation 63
9.1 Start with the right use cases . . . 64
9.2 Treat this as a standardization pattern, not a one-off feature . . . 65
9.3 Prefer the least-privileged rotation model . . . 65
9.4 Onboard conservatively . . . 66
9.5 Make host trust explicit . . . 66
9.6 Design around the real platform dependencies . . . 67
9.7 Separate rotation control from credential consumption . . . 67
9.8 Use manual rotation to prove readiness, then automate . . . 68

9.9 Make offboarding a standard workflow . . . 68
9.10 Plan for failure before you expand . . . 69
9.11 Roll out by account class, not by host count . . . 69
9.12 Reference materials . . . 70

10 Vault Agent templates 70
10.1 Understanding Vault Agent templates . . . 70
10.2 Getting Started with Vault Agent Templates . . . 71
10.2.1 Template language overview . . . 71
10.2.2 Environment variables . . . 73
10.2.3 Application memory . . . 74
10.2.4 PKI certificates . . . 75
10.3 Integration Strategies for Applications . . . 76
10.4 Best practices for using HashiCorp Vault Agent templates in applications . . . 77
10.4.1 Always use least-privilege access . . . 77
10.4.2 Error Handling . . . 77
10.5 Operational Best Practices . . . 78
10.5.1 Monitoring and Logging . . . 78
10.5.2 Common Issues and Solutions . . . 78
10.6 Additional Resources . . . 79

11 Consume secrets in Kubernetes 79
11.1 Vault Secrets Operator (VSO) . . . 79
11.2 Vault Agent Sidecar Injector . . . 80
11.3 CSI Provider . . . 81
11.4 Usage Scenarios . . . 82
11.4.1 Scenario 1 . . . 82
11.4.2 Scenario 2 . . . 82
11.5 Additional References . . . 83

12 Changelog 83
12.1 2026-08 . . . 83
12.2 2026-07 . . . 83
12.3 2026-06 . . . 83
12.4 2026-01 . . . 83
12.5 2025-12 . . . 84

12.6 2025-07 84
12.7 2025-06 84
12.8 2025-05 84
12.9 2025-04 84
12.10 2024-02 84

13 Credits **84**

1 Introduction

Note

These guides were reorganized in August 2026. The previous Solution Design Guide and Operating Guides (Adoption, Standardization, Scaling) have been replaced with three role-based guide types: Installation, Administration, and User. Read the HVD 2.0 release notes for full details on what changed and where to find the content you need.

1.1 Why use HashiCorp Validated Designs?

HashiCorp Validated Designs (HVD) provide practitioners with opinionated guidance for achieving production-grade deployments of HashiCorp products. These designs are purpose-built for delivering foundational use cases, with a baseline level of architectural and operational maturity. They draw on the field experiences of Solutions Engineers and Solutions Architects working with customers across a wide range of environments and organizational requirements.

Each guide provides access to an opinionated reference architecture, including key design decisions and the rationale behind them. Where applicable, guides identify modular design components that you can adjust to align with organizational or regulatory requirements without compromising the overall integrity of the implementation. For many deployments we include Terraform modules to automate large portions of infrastructure provisioning and software installation.

1.2 Audience

This User Guide is for application teams, service owners, and other Vault consumers who use Vault features after platform operators have deployed and configured the service. It focuses on consuming secrets, credentials, certificates, and cryptographic services through approved workflows.

For deployment and first-run configuration, use the Vault Installation Guide for virtual machines or the Vault Installation Guide for OpenShift. For operator setup and ongoing management of Vault features, use the Vault Administration Guide.

Note

Operator tasks such as authentication-method configuration, policy design, secrets-engine enablement, audit-log configuration, backup and restore, disaster recovery, and upgrades are covered in the Installation and Administration guides.

1.3 Use cases covered

This guide covers the Vault capabilities that consumers use once platform operators have configured the service:

Use case	Summary
Static secrets	Retrieve and consume approved static secrets from Vault.
Secrets Sync	Consume secrets through synchronized external destinations configured by operators.
Dynamic secrets	Use short-lived credentials issued by Vault for applications and services.
PKI certificates	Request and renew Vault-issued certificates, including Vault Agent workflows.
Key lifecycle management	Use key management and KMIP workflows after operators configure engines and roles.
Transit and Transform	Use Vault cryptographic and data-protection services for encryption, tokenization, and transformation workflows.
RHEL local account password rotation	Use Vault's OS secrets engine to govern rotation of long-lived RHEL local accounts, such as break-glass and legacy credentials.
Vault Agent templates	Render Vault secrets into application configuration files and runtime environments.

Use case	Summary
Kubernetes secrets consumption	Consume Vault secrets from Kubernetes workloads using approved integration patterns.
Performance replication	Consume Vault from deployments that use performance replication. This section needs further authoring.

1.4 Prerequisites

Before using this guide, confirm that your platform team has provided the Vault address, authentication method, namespace or mount path, and policy grants required for your workload.

2 Static secrets

> **Note**
>
> Static secrets are configured and managed by platform operators. See the Administration Guide: Static secrets management.

2.1 Consumption of secrets

At this point, you should have one human auth method (OIDC or LDAP) and one machine auth method (AWS or AppRole) enabled, allowing both humans and machines to authenticate to Vault. You have also populated some static secrets in the KV v2 secrets engine and created policies to enforce permissions on which identities can access specific secrets. The next step is to consider how to deliver these secrets from Vault to the application.

2.1.1 Secret consumption patterns

There are two patterns for your applications to fetch secrets from Vault: direct API integration or via Vault Agent.

Direct API integration requires that your application make API calls to authenticate with Vault and access secrets. To do this, you could either write custom integration code or depend on a third party client library. Regardless of which option you choose, it requires that your application be Vault-aware, meaning that you need to write and maintain application code to enable your application to communicate with Vault. For some applications, this is not a problem and may actually be the preferred method. For example, if you have only a handful of applications or you want to keep strict, customized control over the way each application interacts with Vault, you might be fine with the added overhead of maintaining that code and making sure a member of each application's team is trained to understand Vault and that code.

In other situations — typically in large enterprises — updating each application's code base could be a monumental task or simply a non-starter for several reasons.

- If your organization has hundreds or thousands of applications, you may not have the time, resources, or expertise to update and maintain Vault integration code in every application.
- Your organization may not permit the teams deploying some applications to add the Vault integration code, or any code. For example, certain legacy applications may be too brittle to allow the addition of Vault integration code.
- The applications (secrets consumers) and the systems (secrets producers) are, in many cases, managed by different teams. This makes coordinating the maintenance of Vault integration code into a clean workflow very difficult.
- Some teams deploy third-party applications that are not owned by the organization, and therefore it is not possible to add Vault integration code.

For those situations, we recommend a much more scalable and simpler experience — using the Vault Agent or Vault Proxy. Vault Agent can obtain secrets and provide them to applications, and Vault Proxy can act as a proxy between Vault and the application, optionally simplifying the authentication process and caching requests.

> **Note**
>
> Vault Proxy is a new feature introduced in Vault 1.14. Vault Agent has the capability to act as an API Proxy for Vault but this feature will be deprecated in future release. See this table comparing the capabilities between Vault Agent and Vault Proxy.

2.1.2 Vault Agent

We briefly introduced Vault Agent earlier in this document when we discussed how Vault solves the secret zero problem. Vault agent is a client daemon that uses the same Vault binary as the Vault server, except that it is run with the `vault agent` command. Here we will explore the features offered by the Vault Agent.

- Auto-Auth - Automatically authenticate to Vault and manage the token renewal process.
- Templating - Allows rendering of user-supplied templates by Vault Agent, using the token generated by the auto-auth step.
- Caching - Allows client-side caching of responses containing newly created tokens and responses containing leased secrets generated off of these newly created tokens. The agent also manages the renewals of the cached tokens and leases.
- Process Supervisor Mode - Runs a child process with Vault secrets injected as environment variables.
- Windows Service - Allows running the Vault Agent as a Windows service.

Auto-auth

Vault is built on zero trust principles, so applications that need to fetch secrets from Vault are required to make authenticated requests by providing a Vault token. Vault Agent can make the authentication process for obtaining a Vault token easy and transparent to applications.

When Vault Agent starts up, it automatically authenticates with Vault and receives a Vault token via a process known as auto-auth. The token can then be written to disk to any number of file sinks - a file sink is a local file or a file mapped via some other process (NFS, Gluster, CIFs, etc.). In addition to file permissions, the token can be protected through Vault's response-wrapping mechanism and encryption. Applications can then use this token to request secrets from Vault.

Vault Agent also manages the lifecycle of the Vault token that is received from auto-auth. This includes timely renewal of the tokens and re-authentication if a token can no longer be renewed. The sink is updated with the new token value whenever re-authentication occurs.

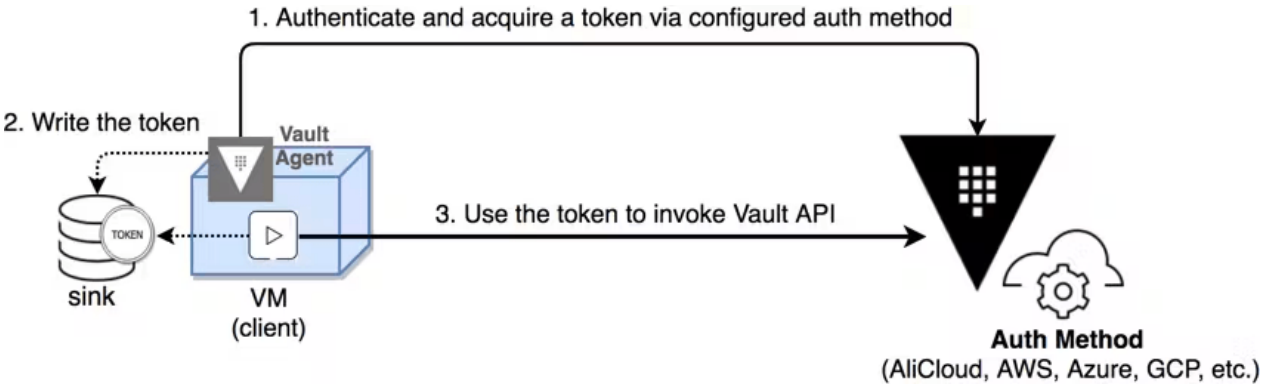


Figure 1: Vault Agent auto auth

The client can simply retrieve the token from the sink and connect to Vault using the token. This simplifies client integration since the Vault agent handles the login and token refresh logic.

To use this feature, run the Vault binary in agent mode (`vault agent -config=<config file path>`) on the client. The agent configuration file must specify the auth method and sink locations that dictate where the token should be written on the filesystem.

Templating

Vault Agent supports the ability to render static and dynamic secrets from Vault by using templates that describe how and where to render those secrets on the local system. Applications or service owners define these templates using Consul Template markup and then the secrets are consumed easily by the applications, eliminating the need for additional formatting and parsing. A common use case for templating is formatting credentials from Vault's Database Secrets Engine to be rendered as database connection strings for applications to consume directly. An example of this approach is demonstrated in the tutorial here.

After the Vault Agent starts up and authenticates with Vault, it can be set up to retrieve secrets needed for an application. It writes these secrets to disk through the templates defined in the agent configuration, which the application can then use as needed. Review the Vault agent templates tutorial.

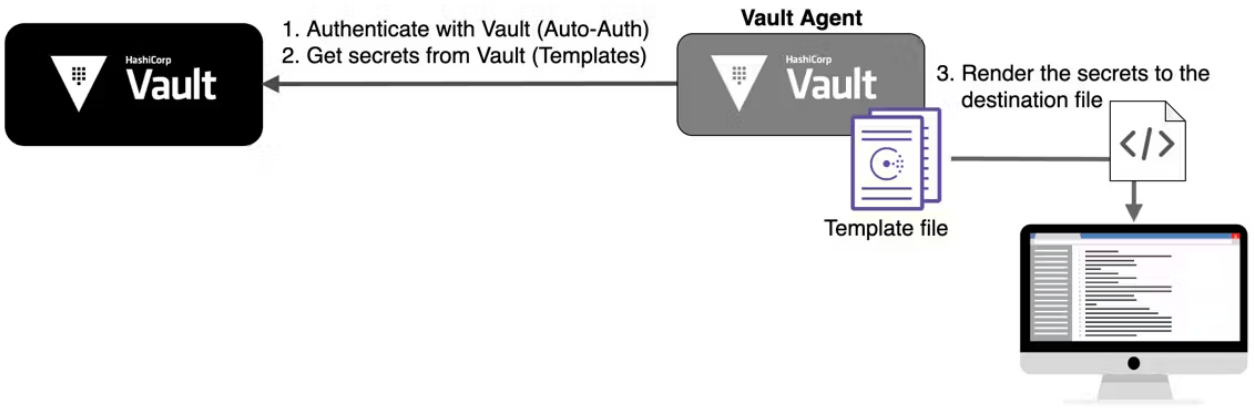


Figure 2: Vault Agent templating

Caching

When Vault is controlling access to all of your secrets, it is very important to ensure that Vault's performance scales as consumption grows, so it can handle the corresponding load. The Vault Agent supports the ability to proxy and cache requests to Vault as well as responses from Vault. This helps reduce the number of requests sent to Vault servers at one time, reducing the peak load on Vault servers.

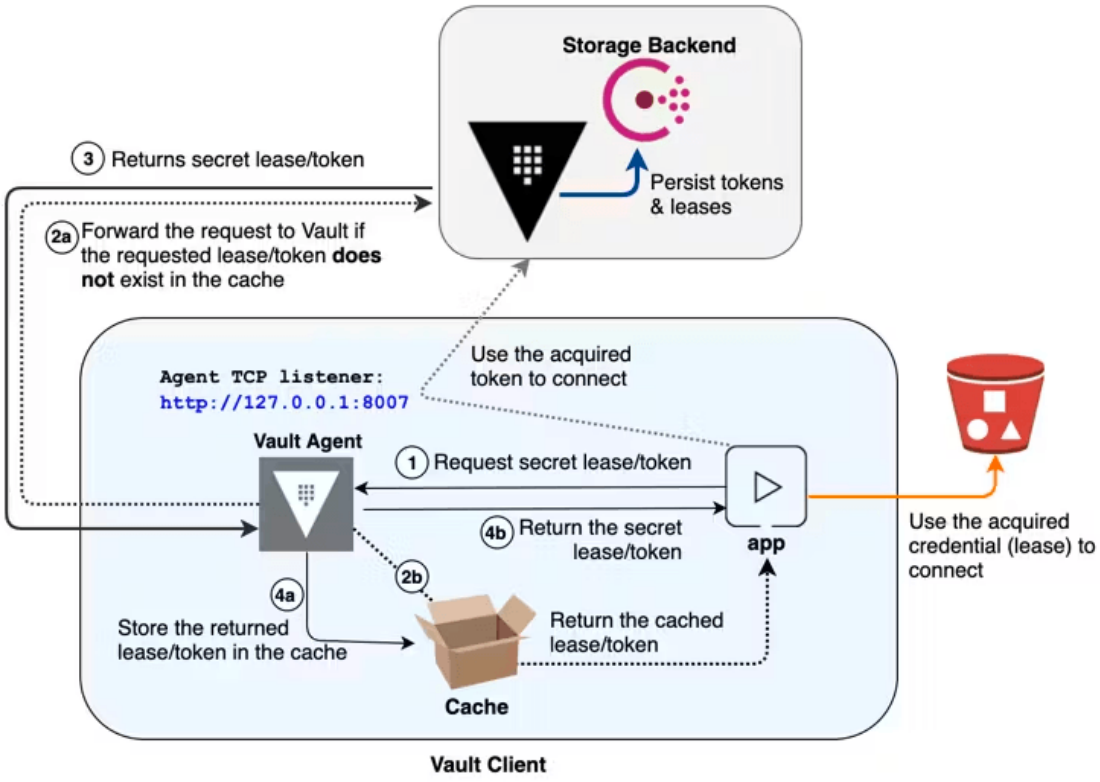


Figure 3: Vault Agent Caching

Reference Materials:

- Vault agent caching documentation
- Vault agent caching tutorial

Process supervisor mode

Vault Agent's process supervisor mode allows secrets to be injected into a process via environment variables using Consul Template markup.

It is common for applications to load certain configurations from environment variables. When the application starts up, it reads environment variables and uses them to initialize connections to various databases, message queues, or other services. Often, these initialization parameters contain secrets such as database credentials, certificates, and API keys.

In this mode, Vault Agent will start the application as a child process with the environment variables populated using the configured secrets. The Vault Agent will act as a process supervisor to the application and will restart it when the configured static secrets change.

Reference Materials:

- Vault agent's process supervisor mode documentation
- Vault Agent - secrets as environment variables tutorial

Windows service

Vault Agent can be run as a Windows service. In order to do this, Vault Agent must be registered with the Windows Service Control Manager. After Vault Agent is registered, it can be started like any other Windows service.

2.1.3 Vault proxy

The Vault Proxy offers an alternative integration method for Vault clients. It is designed to simplify the integration workflow for clients that need to make direct API calls to Vault.

The Vault Proxy acts as an API proxy for Vault. Any requests sent to the Vault Proxy will be forwarded to the Vault server, and these requests can be configured to optionally allow or force the automatic use of the auto-auth token, simplifying the authentication process. It also allows client-side caching of responses to reduce the load to the Vault server.

Unlike Vault Agent templating, clients must be able to integrate with Vault's APIs in order to use Vault Proxy. See this table for a comparison of the capabilities between Vault Agent and Vault Proxy.

Reference Materials:

- Vault Proxy documentation

2.1.4 The Vault Agent advantage

The Vault Agent simplifies the zero trust adoption journey by reducing the effort and time needed by your application teams to integrate with Vault. Applications can easily integrate with the agent,

which handles authentication into Vault along with the fetching, rendering, and lifecycle management of application secrets and the caching of these secrets to reduce the load on Vault.

The diagram below shows that as tasks such as fetching the initial secrets, secrets renewal, secrets rendering, templating, and caching become more Vault agnostic from the perspective of your application, they become easier to manage at scale.

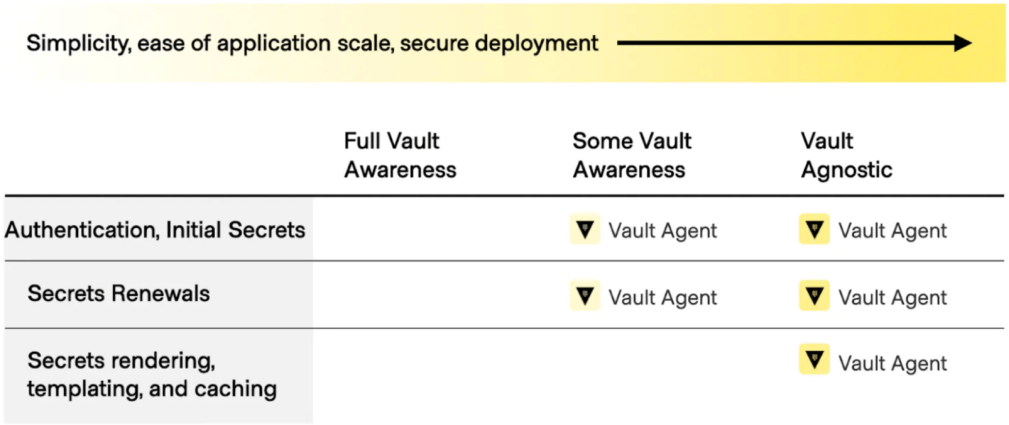


Figure 4: Vault Agent Advantage

2.1.5 Vault Agent best practices

Deployment pattern

For each application that needs access to Vault, we recommend deploying one agent per application. This means that each application has its own login and identity with Vault, which aligns with the practice of least privilege. When using machine identity authentication (e.g. AWS, GCP, Azure), you should deploy a Vault agent for each host. If multiple applications share a host, use separate agent instances for each application with an auth method like AppRole that is not tied to the machine's identity.

Telemetry

The agent supports and collects various runtime metrics about its performance, the auto-auth and the cache status. We recommend that you enable telemetry and send these metrics to a monitoring platform. See the list of available metrics here.

Integration pattern

How you actually integrate Vault Agent into your application deployment workflow will vary based on several factors. Ask yourself the following questions.

- What is the lifecycle of my application? Is it more ephemeral or long-lived?
- What are the lifecycles of my authentication tokens? Are they long-lived and do they require continual renewal to show liveliness of a service or do you want to enforce periodic re-authentications?
- Do I have a group of applications running on my host which each need their own token? Can I use a native authentication capability (e.g. AWS IAM, K8s, Azure MSI, Google Cloud IAM, etc.)?

The answers to these questions will help you decide if Vault Agent should run as a daemon or as a prerequisite of a service configuration. Take for example the following Systemd service definition for running Nomad:

```
[Unit]
Description=Nomad Agent
Requires=consul-online.target
After=consul-online.target
[Service]
KillMode=process
KillSignal=SIGINT
Environment=VAULT_ADDR=http://active.vault.service.consul:8200
Environment=VAULT_SKIP_VERIFY=true
ExecStartPre=/usr/local/bin/vault agent -config /etc/vault-agent.d/vault-agent.hcl
ExecStart=/usr/bin/nomad-vault.sh
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
RestartSec=2
StartLimitBurst=3
StartLimitIntervalSec=10
LimitNOFILE=65536
[Install]
WantedBy=multi-user.target
```

Notice the `ExecStartPre` directive that runs Vault Agent before the desired service starts. Here, we want to run the Vault agent as a prerequisite before Nomad starts up. To prevent Vault agent from running as a daemon, you can enable the exit_after_auth flag to instruct the agent to exit after a single successful authentication and all templates rendered.

The service startup script expects a Vault token value set as shown in the `/usr/bin/nomad-vault.sh` startup script:

```
#!/usr/bin/env bash
if [ -f /mnt/ramdisk/token ]; then
exec env VAULT_TOKEN=$(vault unwrap -field=token $(jq -r '.token' /mnt/ramdisk/token)
    ) \
/usr/local/bin/nomad agent \
-config=/etc/nomad.d \
-vault-tls-skip-verify=true
else
echo "Nomad service failed due to missing Vault token"
exit 1
fi
```

Applications which expect Vault tokens typically look for a VAULT_TOKEN environment variable. Here, you are using Vault Agent auto-auth to get a token and write it out to a RAM disk and as part of the Nomad startup script. You read the response-wrapped token from the RAM disk, and save it to your VAULT_TOKEN environment variable before starting Nomad.

Similarly, you can use Vault Agent templating to generate application configuration files as part of your application startup script. This would enable your application to be completely agnostic from Vault. For example, the template below renders the application settings for a .NET Core application. It contains most of the default configuration for an ASP.NET Core application but creates a `ConnectionString` based on the database username and password from Vault using the database secrets engine.

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    {{- with secret "projects-api/database/creds/projects-api-role" }}
    "Database": "Server=.;Database=HashiCorp;user id={{ .Data.username }};password={{
         .Data.password }}"
    {{- end }}
  }
}
```

Vault Agent will update the application settings each time Vault rotates the database username and password. ASP.NET Core allows live reload of the application each time the application settings file changes. This makes your database credentials to be short-lived and significantly reduces its vulnerability. If your application does not support live reload of application configurations, you can utilize the exec command to execute a command to restart the application each time the template changes. For more details on how to use the Vault agent to generate application configurations, see the Vault Agent with .NET Core tutorial.

2.1.6 Configure the Vault Agent

In this section, you will configure the Vault Agent to consume secrets from the KV v2 secrets engine.

The content in this section assumes that you have completed the configurations in the previous sections. Before you begin, you should have:

- Configured either the OIDC or LDAP auth method for human access.
- Configured either the AWS or AppRole auth method for machine access.
- Enabled the KV v2 secrets engine and populated it with some secrets.
- Configured policies to allow applications to read its required static secrets.

Vault Agent and AWS auth

Before you begin, ensure that you have configured the AWS auth method and tested that you can log in successfully from an EC2 instance.

Step 1: Create the Vault Agent configuration file

Log in to your EC2 instance and create a Vault Agent configuration called `vault-agent-template.hcl` using the example below.

The auto_auth block uses the aws auth method enabled at the auth/aws path on the Vault server. The namespace value specifies where your aws auth method is mounted, replace it with the name of your tenant namespace. The Vault Agent will use the `billing-service-dev` role to authenticate.

The template block configures Vault agent templating. The `content` parameter embeds the content of a consul-template directly into the configuration file rather than supplying the source path

to the template file. The `destination` parameter tells the agent where the output will be written to. See a configuration example here where the source path is provided for the template file.

Finally, the vault block configures how the Agent connects to the remote Vault server.

```
pid_file = "./pidfile"

auto_auth {
  method "aws" {
    namespace = "<tenant namespace>"
    mount_path = "auth/aws"

    config = {
      type = "iam"
      role = "billing-service-dev"
    }
  }
}

template {
  contents = "{{ with secret \"secret/billing-service\" }}{{ .Data.data.foo }}{{ end
     }}"
  destination = "/home/vault-user/render-content.txt"
}

vault {
  address = "https://dev.vault.example.com:8200"
  ca_cert = "/opt/vault/tls/ca.pem"
}
```

Step 2: Start the Vault Agent

Execute the following command to start the agent.

```
$ vault agent -config=./vault-agent-template.hcl
```

You should see the following.

```
2023-10-17T19:31:51.080Z [INFO]  agent.exec.server: starting exec server
2023-10-17T19:31:51.081Z [INFO]  agent.exec.server: no env templates or exec config,
    exiting
2023-10-17T19:31:51.082Z [INFO]  agent.auth.handler: starting auth handler
2023-10-17T19:31:51.082Z [INFO]  agent.auth.handler: authenticating
2023-10-17T19:31:51.084Z [INFO]  agent.sink.server: starting sink server
2023-10-17T19:31:51.085Z [INFO]  agent.template.server: starting template server
2023-10-17T19:31:51.085Z [INFO] (runner) creating new runner (dry: false, once: false
    )
2023-10-17T19:31:51.086Z [INFO] (runner) creating watcher
2023-10-17T19:31:51.138Z [INFO]  agent.auth.handler: authentication successful,
    sending token to sinks
2023-10-17T19:31:51.139Z [INFO]  agent.auth.handler: starting renewal process
2023-10-17T19:31:51.140Z [INFO]  agent.template.server: template server received new
    token
2023-10-17T19:31:51.140Z [INFO] (runner) stopping
2023-10-17T19:31:51.140Z [INFO] (runner) creating new runner (dry: false, once: false
    )
2023-10-17T19:31:51.140Z [INFO] (runner) creating watcher
2023-10-17T19:31:51.143Z [INFO] (runner) starting
2023-10-17T19:31:51.152Z [INFO]  agent.auth.handler: renewed auth token
2023-10-17T19:31:51.175Z [INFO] (runner) rendered "(dynamic)" => "/home/ssm-user/
    render-content.txt"
^C==> Vault Agent shutdown triggered
2023-10-17T19:31:58.456Z [INFO]  agent.exec.server: exec server stopped
2023-10-17T19:31:58.456Z [INFO] (runner) stopping
2023-10-17T19:31:58.456Z [INFO]  agent.template.server: template server stopped
2023-10-17T19:31:58.456Z [INFO]  agent.auth.handler: shutdown triggered, stopping
    lifetime watcher
2023-10-17T19:31:58.456Z [INFO]  agent.auth.handler: auth handler stopped
2023-10-17T19:31:58.456Z [INFO]  agent.sink.server: sink server stopped
2023-10-17T19:31:58.456Z [INFO]  agent: sinks finished, exiting
```

Now navigate to the directory where you have configured the agent to output the templated secrets. You should see that the content of the file contains the value of the secret.

```
$ cat render-content.txt
bar
```

Step 3: Render secrets as environment variables

Another way to provide secrets to your application is to inject them as environment variables. Vault Agent's process supervisor mode allows Vault secrets to be injected into a process via environment variables using Consul Template markup.

Create an agent configuration called `vault-agent-envar.hcl` using the example below.

```
pid_file = "./pidfile"

auto_auth {
  method "aws" {
    namespace = "<tenant namespace>"
    mount_path = "auth/aws"

    config = {
      type = "iam"
      role = "billing-service-dev"
    }
  }
}

template_config {
   static_secret_render_interval = "1m"
   exit_on_retry_failure         = true
}

vault {
  address = "https://dev.vault.example.com:8200"
  ca_cert = "/opt/vault/tls/ca.pem"
}

env_template "MY_SECRET" {
   contents             = "{{ with secret \"secret/data/billing-service\" }}{{ .Data.
       data.foo }}{{ end }}"
   error_on_missing_key = true
}

exec {
   command                   = ["./kv-demo.sh"]
   restart_on_secret_changes = "always"
   restart_stop_signal       = "SIGTERM"
}
```

The `template_config` block has configuration entries that affect all templates. The static_secret_render_interval entry configures how often Vault Agent Template should render non-lease secrets such as KV v2. Here we are telling the Agent to re-render the template every minute. The exit_on_retry_failure entry configures Vault Agent to exit after it has exhausted its number of template retry attempts due to failures. The number of retries can be configured using the retry stanza, but the default number of retries is 12.

The env_template block maps the template specified in the `contents` to the environment variable name in the title of the block. In the example, we are injecting an environment variable called `MY_SECRET` with the value of the secret `foo` at `secret/data/myapp`. The `error_on_missing_key` configuration tells the Agent to exit with an error if the secret key being accessed does not exist.

The exec block configures how the child process is started. The command entry specifies the command to start the child process. restart_on_secret_changes controls whether the agent will restart the child process on secret changes. For example, if the static secret is updated (on static_secret_render_interval), then the agent will restart the child process. The restart_stop_signal is the signal that the agent will send to the child process when a secret has been updated and the process needs to be restarted. The process has 30 seconds after this signal is sent until `SIGKILL` is sent to force the child process to stop.

Create a simple shell script called `kv-demo.sh` which will display the value of the environment variable named `MY_SECRET`.

```
#!/bin/sh

echo
echo "==== DEMO APP ===="
echo "MY_SECRET    = ${MY_SECRET}"

sleep 100
```

Step 4: Start the Vault Agent with new configuration

Start the agent by executing the following command.

```
$ vault agent -config=./vault-agent-envar.hcl
```

You should see the following.

```
2023-10-17T19:35:45.423Z [INFO]  agent.exec.server: starting exec server
2023-10-17T19:35:45.423Z [INFO] (runner) creating new runner (dry: true, once: false)
2023-10-17T19:35:45.424Z [INFO] (runner) creating watcher
2023-10-17T19:35:45.424Z [INFO]  agent.auth.handler: starting auth handler
2023-10-17T19:35:45.424Z [INFO]  agent.auth.handler: authenticating
2023-10-17T19:35:45.424Z [INFO]  agent.sink.server: starting sink server
2023-10-17T19:35:45.424Z [INFO]  agent.template.server: starting template server
2023-10-17T19:35:45.424Z [INFO]  agent.template.server: no templates found
2023-10-17T19:35:45.469Z [INFO]  agent.auth.handler: authentication successful,
    sending token to sinks
2023-10-17T19:35:45.469Z [INFO]  agent.auth.handler: starting renewal process
2023-10-17T19:35:45.470Z [INFO]  agent.exec.server: exec server received new token
2023-10-17T19:35:45.470Z [INFO] (runner) stopping
2023-10-17T19:35:45.470Z [INFO] (runner) creating new runner (dry: true, once: false)
2023-10-17T19:35:45.470Z [INFO] (runner) creating watcher
2023-10-17T19:35:45.470Z [INFO] (runner) starting
2023-10-17T19:35:45.480Z [INFO]  agent.auth.handler: renewed auth token
2023-10-17T19:35:45.494Z [INFO] (runner) rendered "(dynamic)" => "MY_SECRET"
2023-10-17T19:35:47.495Z [INFO]  agent.exec.server: [INFO] (child) spawning: ./kv-
    demo.sh

==== DEMO APP ====
MY_SECRET    = bar
```

Now change the value of the static secret in Vault. The agent will re-render the secret every minute, and if the value of the secret changes, it will restart the child process and the updated secret value will be shown.

```
2023-09-26T19:32:36.727Z [INFO] (runner) rendered "(dynamic)" => "MY_SECRET"
2023-09-26T19:32:38.727Z [INFO]  agent.exec.server: stopping process: process_id
    =118363
2023-09-26T19:32:38.728Z [INFO]  agent.exec.server: [INFO] (child) stopping process
2023-09-26T19:32:38.728Z [INFO]  agent.exec.server: [INFO] (child) spawning: ./kv-
    demo.sh

==== DEMO APP ====
MY_SECRET    = bazzzz
```

Vault Agent and AppRole auth

Before you begin, ensure that you have configured the AppRole auth method and tested that you can login successfully.

Step 1: Create the Vault Agent configuration file

Login to the VM where the application is deployed and create a Vault Agent configuration called `vault-agent-template.hcl` using the example below.

The auto_auth block uses the approle auth method enabled at the auth/approle path on the Vault server. The namespace value specifies where your approle auth method is mounted, replace it with the name of your tenant namespace. The role_id_file_path is the path to the file with the Role ID and the secret_id_file_path is the path to the file with the Secret ID. By default, the Vault agent will delete the Secret ID file after it is read. The secret_id_response_wrapping_path tells the Agent that the Secret ID is response-wrapped and should be verified against the response wrapping path set here. Using a response-wrapped Secret ID is recommended as discussed in the AppRole Best Practice section.

The template block configures Vault agent templating. The `source` parameter specifies the source path to the template file. The `destination` parameter tells the agent where the output will be written to.

Finally, the vault block configures how the Agent connects to the remote Vault server.

```
pid_file = "./pidfile"

auto_auth {
 method {
   type = "approle"
   namespace = "demo"
   config = {
     role_id_file_path = "/home/vault/app1_role_id"
     secret_id_file_path = "/home/vault/app1_secret_id"
     secret_id_response_wrapping_path = "auth/approle/role/billing-service-dev/secret
         -id"
   }
 }
}

template {
 source = "/home/vault/kv.tpl"
 destination = "/home/vault/kv.html"
}

vault {
 address = "https://dev.vault.example.com:8200"
 ca_cert = "/opt/vault/tls/ca.pem"
}
```

Step 2: Create the template file

Create the template file referenced in the agent configuration. The example below creates an HTML file by fetching the secret `foo` from the path `secret/billing-service`. You can find more information on the consul-templating language here.

```
tee /home/vault/kv.tpl <<EOF
{{ with secret "secret/billing-service" }}
<html>
<head>
</head>
  <body>
    <li><strong>foo</strong> &#58; {{ .Data.data.foo }}</li>
</body>
</html>
{{ end }}
EOF
```

Step 3: Start the Vault Agent

Start the agent by executing the following command.

```
$ vault agent -config=./vault-agent-template.hcl
```

You should see the following.

```
2023-10-17T20:51:51.119Z [INFO]  agent.exec.server: starting exec server
2023-10-17T20:51:51.119Z [INFO]  agent.exec.server: no env templates or exec config,
    exiting
2023-10-17T20:51:51.119Z [INFO]  agent.auth.handler: starting auth handler
2023-10-17T20:51:51.119Z [INFO]  agent.auth.handler: authenticating
2023-10-17T20:51:51.120Z [INFO]  agent.sink.server: starting sink server
2023-10-17T20:51:51.120Z [INFO]  agent.template.server: starting template server
2023-10-17T20:51:51.120Z [INFO] (runner) creating new runner (dry: false, once: false
    )
2023-10-17T20:51:51.122Z [INFO] (runner) creating watcher
2023-10-17T20:51:51.189Z [INFO]  agent.auth.handler: authentication successful,
    sending token to sinks
2023-10-17T20:51:51.189Z [INFO]  agent.auth.handler: starting renewal process
2023-10-17T20:51:51.194Z [INFO]  agent.template.server: template server received new
    token
2023-10-17T20:51:51.194Z [INFO] (runner) stopping
2023-10-17T20:51:51.198Z [INFO] (runner) creating new runner (dry: false, once: false
    )
2023-10-17T20:51:51.199Z [INFO] (runner) creating watcher
2023-10-17T20:51:51.203Z [INFO] (runner) starting
2023-10-17T20:51:51.211Z [INFO]  agent.auth.handler: renewed auth token
2023-10-17T20:51:51.237Z [INFO] (runner) rendered "/vault-agent/kv.tpl" => "/vault-
    agent/kv.html"
```

Now navigate to the directory where you have configured the agent to output the templated secrets. You should see that the content of the file contains the value of the secret.

```
$ cat kv.html

<html>
<head>
</head>
 <body>
   <li><strong>foo</strong> &#58; bar</li>
</body>
</html>
```

3 Secrets Sync

> **Note**
>
> Secrets Sync destinations and policies are configured and managed by platform operators. See the Administration Guide: Secrets sync management.

3.1 Example scenario

Consider this situation: You need to provide credentials, such as database credentials or API keys, to multiple applications provisioned across various cloud providers. These applications already consume secrets from the secrets manager within their respective cloud environments, or the credentials are hard-coded into the application code. Each application development team needs to manage their own secrets while also having them stored centrally for organizational auditing. Each development team is assigned a tenant namespace in your organization's Vault Enterprise deployment.

The recommended organizational layout would be as follows:

- Root administrators
    - Enable the secrets sync feature.
    - Set up tenant namespaces.
- Application A
    - Resides in AWS and GCP.
    - Managed by application development team A.
- Application B
    - Resides in Azure.
    - Managed by application development team B.
- Application development team A
    - Responsible for the development and integration of application A.
    - Namespace users: Can read and update K/V secrets and associate these secrets with a sync destination.
    - Namespace admins: Have elevated admin rights and can set up and configure sync destinations.
- Application development team B
    - Responsible for the development and integration of application B.
    - Namespace users: Can read and update K/V secrets and associate these secrets with a sync destination.

- Namespace admins: Have elevated admin rights and can set up and configure sync destinations.

This structure allows each development team to manage their secrets independently, while ensuring centralized auditing and control through Vault Enterprise.

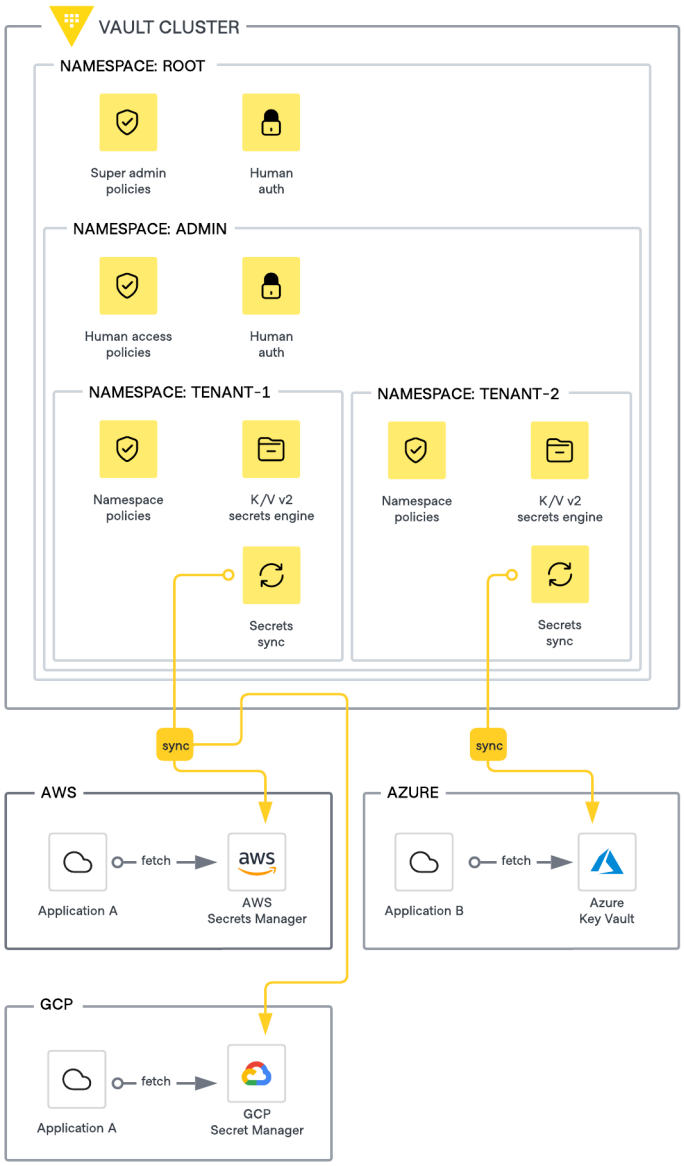


Figure 5: Secrets sync namespace design

4 Dynamic secrets

> **Note**
>
> Dynamic secrets engines and roles are configured and managed by platform operators. See the Administration Guide: Dynamic secrets management.

4.1 Application considerations for dynamic credentials

Applications will need to be aware of password rotation in certain cases, especially for long-running jobs and applications.

For example, in the case of Spring Boot applications, you would do the following:

1. **Configure Spring Cloud Vault:** Set up Spring Cloud Vault in your application.properties or application.yml file, specifying the necessary configurations to connect to Vault and access the dynamic secrets.
2. **Enable refresh scope:** For any beans that should be refreshed when the configuration changes, use the @RefreshScope annotation. This might include your DataSource bean or any configuration properties class that holds your database credentials.
3. **Enable Actuator refresh endpoint:** Make sure the Actuator's refresh endpoint is enabled and exposed. In Spring Boot 2.x, this might involve setting properties in your application.properties or application.yml, like
   management.endpoints.web.exposure.include: refresh
4. **Create a refresh mechanism:** Implement a mechanism to call the /actuator/refresh endpoint whenever the credentials in Vault are rotated. This could be a scheduled task within your application, an external script, or a webhook triggered by Vault (if Vault is configured to send notifications on credential rotation).
5. **Secure the Actuator endpoints:** Ensure that your Actuator endpoints, including the /refresh endpoint, are secured to prevent unauthorized access. Use Spring Security to restrict access to these endpoints.
6. **Refresh DataSource credentials:** Ensure your DataSource or the component that manages the database connection is designed to handle configuration changes dynamically. This might involve using a DataSource proxy or custom logic to reinitialize the DataSource with new credentials upon refresh. Here is a simplified example demonstrating how you might configure a DataSource bean to use dynamic credentials that can be refreshed via Actuator:

```
@Configuration
public class DataSourceConfig {
@Bean
@RefreshScope
public DataSource dataSource(
@Value("${spring.datasource.url}") String url,
@Value("${spring.datasource.username}") String username, @Value("${spring.datasource.
    password}") String password) {
return DataSourceBuilder.create()
.url(url)
.username(username)
.password(password)
.build();
}
}
```

5 PKI certificates

Note

Vault PKI hierarchy, roles, policies, and certificate authority controls are configured and managed by platform operators. See the Administration Guide: PKI certificate management.

5.1 A pattern for automatic certificate renewal

This page includes guidance on the configuration of HashiCorp Vault's PKI secrets engine, CA hierarchy design, and access control best practices. We focus on using Vault Agent for seamless certificate retrieval and renewal, enabling secure and efficient management of ephemeral X.509 certificates. See the related resources section at the end of the page for other consumption workflows.

The primary benefits of this approach include:

- **Enhanced compliance**: Enforce strong, consistent, identity-based security controls and streamline certificate issuance to meet regulatory and organizational policies.
- **Downtime prevention**: Automate certificate renewal and integrate post-rotation actions to maintain secure, uninterrupted service operations.
- **Cost optimization**: Minimize manual effort and administrative overhead by automating routine certificate lifecycle management tasks.

- **Reduced risk**: Mitigate the possibility of certificate or key compromise by enforcing short lifetimes and automating rotation, limiting the impact of leaked secrets.

5.2 Producers and consumers of certificates

We recommend a producer/consumer model for Vault PKI, applying the following roles to improve the security posture of certificate management.

- **Platform operator**: Maintains and scales the Vault platform from an infrastructure perspective. This role may reside within security-focused teams or broader infrastructure platform groups.
- **Secrets producer**: Configures and manages PKI secrets and operational patterns in Vault. Responsibilities include configuration of certificate authorities, setting up certificate issuance policies, defining access controls, and ensuring compliance with applicable regulations or controls. This role may include Vault administrators, PKI specialists, or security engineers.
- **Secrets consumer**: Integrates Vault-issued certificates with various applications. This category includes DevOps engineers, application developers, and service owners who rely on certificates for secure communications.

5.3 Vault Agent for VM-based workloads

Using Vault Agent for certificate management of VM-based workloads.

5.3.1 Introduction to Vault Agent

Applications must interact with the Vault API to automate certificate renewal. In some cases, teams might build and maintain custom automation for authentication, certificate retrieval, renewal timing, and persistence. However, this approach may not scale well in large enterprises with diverse platforms and runtimes, or may be wholly inappropriate for the target application.

Vault Agent simplifies client-side integration with Vault by abstracting authentication, token management, and secrets consumption. For PKI workloads, it automates the certificate lifecycle and provides flexible templating features to render certificate files in formats tailored to application requirements. In addition, Vault Agent can trigger scripted actions after it issues a new certificate, such as reloading a service or calling a webhook.

5.3.2 PKI consumption workflow

This consumption pattern demonstrates automating certificate management for applications with Vault Agent. The diagram below illustrates the architecture of an example application and how Vault Agent interacts with Vault to obtain certificates. In this example, an NGINX server running on an AWS EC2 instance requires a certificate for an HTTPS listener. NGINX, like many other software applications, references certificates and private keys from files on the local machine. Other examples of software solutions that implement this pattern include Apache HTTPD, HAProxy, PostgreSQL, MySQL, MongoDB, Kafka, RabbitMQ, Jetty, Tomcat, and Elasticsearch.

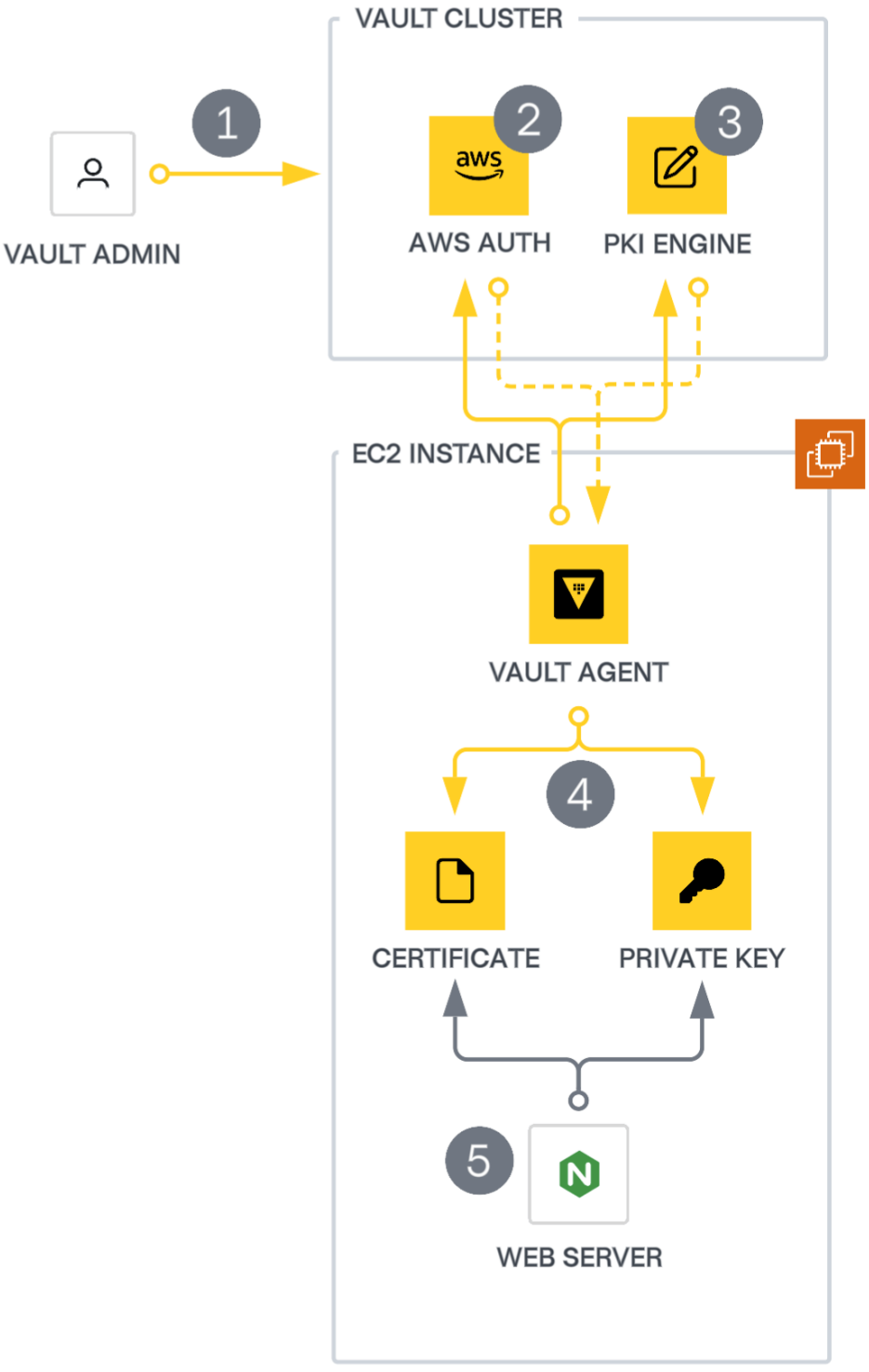


Figure 6: PKI consumption workflow

The workflow for obtaining a certificate consists of five main steps, explained in the diagram:

1. Initial configuration: A Vault administrator must configure an auth method, role, policy, and PKI engine role to allow Vault to authenticate the client and issue a valid certificate. The administrator can use tools such as the Vault CLI, API, or Terraform to perform this configuration. Use Terraform as a best practice.
2. Authentication: Upon startup, the agent automatically authenticates with the Vault cluster and obtains a token for subsequent requests. This example uses the AWS auth method.
3. Certificate retrieval: The agent retrieves a certificate by sending a request to the issuing PKI secrets engine. The secrets engine generates a certificate based on the request and the PKI role configuration and returns the certificate to the Agent.
4. Template rendering: The agent converts the certificate data into the required format for the application and saves it to the filesystem. In this scenario, you have configured the agent to store the certificate and private key in separate files, as is common for most TLS-enabled application servers.
5. Application usage: NGINX can now utilize the rendered certificate and private key by referencing the paths to the agent-rendered files.

Vault Agent continuously monitors and manages the lifecycle of the certificate and key, automating the renewal process and preventing downtime due to an expired certificate. As the expiration date approaches, the agent retrieves a new certificate from the PKI secrets engine, re-renders the certificate and private key to the file system, and performs any necessary steps to notify the application about the new certificate (steps 3-5).

5.3.3 Implementation guide

The guide assumes that you have set up a Vault Enterprise cluster for PKI issuance by following the best practices defined in the Vault HVD. It also presumes that you have implemented the guidance in Validated PKI Architecture, including engine configurations, tenant isolation, CA hierarchy, authentication methods, ACL policies, and application-specific PKI role configurations.

Certificate generation APIs (sign versus issue)

Vault supports two primary methods for obtaining certificates from the PKI secrets engine: `issue` and `sign`. Both endpoints are available to any PKI role, provided the client has appropriate access via ACL policy.

The `issue` endpoint is for fully automated workflows. On request, Vault generates a private key and issues a certificate, returning both to the client. The private key is ephemeral, is not retained by Vault, and must be securely handled by the requesting application. Vault may store the certificate itself if you configure the role to do so.

The `sign` endpoint supports a more traditional CA workflow, where the client generates its own key pair and submits a Certificate Signing Request (CSR) to Vault for evaluation. In this case, Vault signs the CSR and returns only the certificate. The private key remains under the exclusive control of the client and is never seen by Vault.

While both methods are valid, Vault Agent templating requires the issue endpoint. This capability forms the foundation of the agent-based patterns and enables hands-free certificate lifecycle management for a variety of workloads.

Authentication

Vault Agent must authenticate with Vault like any other client. Therefore, the next step is to identify the auth method that the Vault Agent will use to log in and obtain a valid token. In many cases, the Vault Platform Team makes the decision of which auth method to use.

Vault Agent supports the majority of application-oriented auth methods. As with any other use case, use a platform identity source (AWS IAM, Kubernetes service accounts, Azure MSI, and so on) instead of a static credential for Vault authentication. For workloads without a built-in source of identity, we recommend a trusted orchestrator pattern to provide credentials, such as an AppRole secret ID.

Agent installation and configuration

The first step is to install the Vault Agent binary on your application host. In our example, this corresponds to the EC2 instance running NGINX. Ideally, automation would install and configure the agent during workload provisioning. For a detailed tutorial covering agent installation and basic operation, see the Vault Agent and Vault Proxy Quick Start guide.

After installing the Vault Agent, the next step is to construct a valid configuration file that the agent will use when communicating with Vault. The configuration file specifies the Vault cluster address, auth method, and also defines templates for the secrets your application requires.

Auto-authentication

Vault Agent supports automatic authentication. The `auto_auth` block of the configuration file specifies the auth method and any relevant options. You should configure it to utilize the auth method and role defined for your application. While the exact configuration options will differ depending on your chosen auth method, the example below illustrates how you would set it up for AWS IAM authentication.

```
auto_auth {
   method "aws" {
       mount_path = "auth/aws"
       config = {
           type = "iam"
           role = "my-app"
       }
   }
}
```

Note: Auto-auth can also specify token *sinks* that store the Vault token in a local file. Sinks are optional and you should not include them unless your use case requires direct access to a Vault token. In our example, a sink is not required.

PKI template

Vault Agent has powerful templating capabilities, enabling you to render secret data as files or environment variables for applications to use. If a secret changes due to an update or scheduled rotation, the agent will ensure that any templates are re-rendered. This section will discuss best practices for using these templates to manage certificates.

Template configuration

Use `template` blocks to specify individual templates in the agent configuration. The template documentation lists all available configuration options.

```
template {
    source = "/vault-agent/pkiCerts.tmpl"
    destination = "/vault-agent/template-output/pki.data"
}
```

You can specify the template contents either inline through the `contents` field or store them in a file and provide them via the `source` field. Our example assumes you store the source template in a

file at `/vault-agent/pkiCerts.tmpl`. The template renders output to `/vault-agent/template-output/pki.data`. This `destination` file acts as a persistent cache for the `pkiCert` function output and consuming applications do not use it directly.

The `template_config` block specifies global templating options. The most important configuration option for certificates is the `lease_renewal_threshold`. This threshold determines how long the template engine waits to attempt a renewal of the underlying certificate. It is defined as a fraction of the certificate's total lifetime and defaults to 90 percent. Therefore, if your certificate has a TTL of 10 hours, the agent begins its renewal attempts approximately 1 hour before the expiration date. Depending on your configured certificate TTL and preferred practices, you may choose to adjust this buffer. The example below demonstrates changing this to 75 percent of the certificate's lifetime.

```
template_config {
   lease_renewal_threshold = 0.75
}
```

Templating functions

The templating language offers several helper *functions*. The two relevant functions for the PKI use case are `secret` and `pkiCert`. Both can retrieve certificates from Vault, but they differ in how they handle renewals.

The `secret` function is suitable for generic consumption of a variety of Vault secret types. When rendering a certificate using `secret`, the Vault Agent will always fetch a new certificate at startup or during re-authentication, even if the current certificate is valid. This approach may or may not be appropriate for your use case.

The `pkiCert` function manages rendering and renewals by checking the file system for an existing certificate on the target file system. If no certificate exists at the destination path, the agent retrieves and renders a new certificate. If a certificate is already present, the agent examines its expiration date. If the certificate has expired or is past the renewal threshold, a new one replaces it. However, if the certificate is still valid, the existing certificate remains in place and the agent continues to monitor it. Due to the enhanced behavior, we recommend the `pkiCert` function for all certificate management use cases.

Rendering certificates

In the case of our example NGINX application, the rendered certificate file should include the leaf certificate generated by Vault, concatenated with the chain of intermediate certificates used for signing. Note that our example does not include the root certificate, as we expect to distribute anchor certificates out-of-band and install them in device trust stores outside this workflow.

To construct the content of the template source file, we first specify a function (`pkiCert`), an API path (`pki/issue/team-a`), and any necessary or desired parameters that you should include in the request payload, supplied as key-value pairs (for example, `ttl`):

```
{{- with pkiCert "pki/issue/team-a" "common_name=app.tenant-1.example.com" "ttl=14d"
    "remove_roots_from_chain=true" -}}
```

This line of the template, on its own, only defines the request to Vault. It does not generate any output that will be written to the destination file.

Next, we define how the API response from Vault should be interpreted, transformed, and written to the local file system where the agent is running. The `pkiCert` function supports several helper *keys* that simplify this parsing:

- `.Cert`: the certificate body
- `.Key`: the private key
- `.CAChain`: the CA chain defined in the PKI engine configuration

> **Note**
>
> The `.CAChain` field is only available in versions 1.18 or above of Vault Agent. If you are running an older version, see the consul-template documentation for alternative options.

Adding these keys to the template source file writes the corresponding API response values to the configured default template destination. This example template would produce a single file containing the PEM-formatted private key, certificate, and CA chain:

```
{{- with pkiCert "pki/issue/team-a" "common_name=app.tenant-1.example.com" "ttl=14d"
    "remove_roots_from_chain=true" -}}
{{- .Key -}}
{{- .Cert -}}
{{- .CAChain -}}
{{- end -}}
```

The output produced by this template has limited practical utility and likely cannot be used by an application such as NGINX. However, it is important to configure a default template output contain-

ing unique secret data so that the template engine (and `pkiCert` function in particular) establishes a source of comparison when deciding whether to request a new certificate or other secret. This output corresponds to the *cache* file mentioned in the template configuration section.

To generate separate files containing the private key and certificate data as required by NGINX, additional outputs are defined using the `writeToFile` function. This function can write individual response values to specific files, concatenate secret data into existing files, and also set filesystem permissions on these outputs.

Usage of the `writeToFile` function:

```
writeToFile "[output-path]" "owner" "group" "permission-bits"
```

In this example, the private key and certificate are written to distinct files at `/etc/nginx/certs/`, in addition to the default (cache) output of the template. Appropriate filesystem permissions are also enforced.

```
{{- with pkiCert "pki/issue/team-a" "common_name=app.tenant-1.example.com" "ttl=14d"
    "remove_roots_from_chain=true" -}}
{{- .Key -}}
{{- .Cert -}}
{{- .CAChain -}}
{{- .Key | writeToFile "/etc/nginx/certs/private.key" "" "" "0600" -}}
{{- .Cert | writeToFile "/etc/nginx/certs/server.crt" "" "" "0644" -}}
{{- end -}}
```

Since a standard TLS server handshake should include any intermediate certificates needed to construct a valid trust chain, we must also render the CA chain from the API response, concatenating it onto the leaf certificate file at `/etc/nginx/certs/server.crt`. Because the chain is represented as a list object in the API response, we must iterate through the chain using the `range` function and then append those certificates to the output.

This example demonstrates a complete template, suitable for managing certificates for our NGINX application:

```
{{- with pkiCert "pki/issue/team-a" "common_name=app.tenant-1.example.com" "ttl=14d"
    "remove_roots_from_chain=true" -}}
{{- .Key -}}
{{- .Cert -}}
{{- .CAChain -}}
{{- .Key | writeToFile "/etc/nginx/certs/private.key" "" "" "0600" -}}
{{- .Cert | writeToFile "/etc/nginx/certs/server.crt" "" "" "0644" -}}
{{- range .CAChain -}}
{{- . -}}
{{- . | writeToFile "/etc/nginx/certs/server.crt" "" "" "0644" "append" -}}
{{- end -}}
{{- end -}}
```

Certificate metadata

Vault 1.17 introduced the ability to add custom metadata to your certificates, allowing you to associate any valuable context or information with them. Examples of custom metadata include application, certificate owner, contact information, business unit, risk profile, host, and more. The system stores metadata separately from the certificates themselves, enabling you to utilize this feature even if you choose not to store issued certificates.

To configure a PKI role to store metadata, regardless of whether you store certificates, set `no_store_metadata=false`. Note that including metadata in a certificate request forces the request to forward to the leader node since this action constitutes a storage write operation. This carries a performance penalty in the form of latency, similar to that of certificate storage.

You can add certificate metadata at the time of certificate creation by setting the `cert_metadata` input field. The metadata can be in any format you choose, but it must be base64-encoded before sending it to the API. Typically, you will want the metadata in a standard format, such as JSON or YAML, for easier processing. Vault Agent offers several helper methods that you can use to add this formatted metadata.

```
{{- scratch.MapSet "certMetadata" "teamName" "team-a" -}}
{{- scratch.MapSet "certMetadata" "application" "my-app" -}}
{{- scratch.MapSet "certMetadata" "contact/email" "team-a@example.com" -}}
{{- scratch.MapSet "certMetadata" "contact/slack" "#team-a" -}}
{{- $certMetadata := scratch.Get "certMetadata" | explodeMap | toJSON | base64Encode
    -}}
{{- $certMetadataArg := printf "cert_metadata=%s" $certMetadata -}}


{{- with pkiCert "pki/issue/team-a" "common_name=app.tenant-1.example.com" "ttl=14d"
    "remove_roots_from_chain=true" $certMetadataArg -}}
...
```

The preceding template uses the `scratch` helper to construct a map of the custom certificate metadata we want to assign to the certificate. The `/` separator can be used when defining the keys to create nested objects within the metadata. The map is then processed through `explodeMap`, `toJson`, and base64Encode to generate a base64-encoded JSON object.

The template passes the encoded string in as an argument to the `pkiCert` call and the system stores it by certificate serial number. You can see the resulting certificate metadata for our sample template below:

```
$ vault read -field cert_metadata pki/cert-metadata/<serial> | base64 -d | jq
{
  {
  "application": "my-app",
  "contact": {
    "email": "team-a@example.com",
    "slack": "#team-a"
  },
  "teamName": "team-a"
}
```

Integrating with applications

Often, applications need to be notified when new certificate data is available. For example, NGINX does not automatically reload the TLS certificate when it changes on disk. Instead, it continues to use the previously loaded certificate and key until the configuration is reloaded. In this section, we will discuss controlling application behavior based on changes in certificate (or other secret) data.

Post-render commands

Vault Agent enables you to execute arbitrary commands after a template is rendered, including when secrets are modified. To specify a post-render command, use an `exec` block within the template configuration stanza. The `exec` block contains a `command` field where you specify the exact command and its arguments.

The example below demonstrates how to trigger a reload of the Nginx TLS configuration with `nginx -s reload` whenever the certificate is updated. This ensures that the service is always running with a valid certificate, avoiding outages related to expired certificates.

```
template {
    source = "/vault-agent/pkiCerts.tmpl"
    destination = "/vault-agent/template-output/pki.pem"
    exec {
       command = ["nginx", "-s", "reload"]
    }
}
```

Agent deployment and operations considerations

Given that certificates are a crucial component of most web applications, it is essential that Vault Agent be deployed in a resilient manner. If your application is long-running, run the agent in daemon mode (with `exit_after_auth` set to **`false`**) and managing its execution with a service manager, such as `systemd` or Windows Service Control Manager, to ensure it starts automatically on boot and restarts in the event of any unexpected errors.

If the application requiring a certificate runs on a host with other applications that also need secrets or certificates via Vault Agent, you should deploy a single instance of Vault Agent for each application. This enables each application to authenticate individually, helping you maintain least-privilege access principles. This separation further reduces the blast radius of any potential misconfiguration.

Lastly, Vault Agent supports telemetry and logging, both of which should be used. The metrics allow you to monitor its performance, authentication status, and more. Consider ingesting your metrics and logs into your enterprise monitoring solutions to proactively identify issues with agents and prevent certificates from not being rotated due to problems like authentication failures.

6 Key lifecycle management

Note

Vault key management and KMIP engines are configured and managed by platform operators. See the Administration Guide: Key lifecycle management administration.

6.1 Overview

Organizations generate and store vast quantities of sensitive data. Protecting this data from unauthorized access, corruption, or breaches is crucial. Encryption serves as a critical line of defense, safeguarding information by rendering it unreadable without a decryption key. However, the security of encrypted data hinges entirely on the proper management of those keys. This is where encryption key lifecycle management comes into play.

Key lifecycle management encompasses the entire lifecycle of cryptographic keys, from their secure generation and distribution to their controlled usage, storage, archiving, and eventual destruction. Its primary objective is to ensure the confidentiality and integrity of encrypted data by safeguarding keys from cybercriminals, unauthorized individuals, or accidental exposure.

HashiCorp Vault as a KMS solution allows organizations to maintain centralized control of their keys while also providing robust key lifecycle management. We'll explore each Vault key management solution as well as best practices for leveraging Vault's capabilities to generate, store, access, rotate, and audit cryptographic keys. By adopting these best practices, organizations can ensure the security of their encryption keys, safeguarding sensitive data.

6.1.1 Bring Your Own Key (BYOK)

Bring Your Own Key (BYOK) is a key management strategy that empowers organizations to retain control over their encryption keys across various platforms and services. By generating and managing their own keys, organizations can significantly enhance data security and compliance.

The key benefits of a BYOK approach include:

- **Independent Key Generation**: BYOK allows organizations to verify the integrity and security of their encryption keys by independently generating them using trusted entropy sources. This ensures compliance with stringent security and regulatory requirements.

- **Exclusive Key Ownership**: Organizations maintain exclusive ownership of their encryption keys, ensuring complete control over data access and usage. This eliminates reliance on a third party for key management, mitigating potential risks.
- **Disaster Recovery**: By retaining the original copy of the key material outside of the cloud environment, organizations can implement robust disaster recovery plans to protect against data loss due to cloud service disruptions or failures.
- **Seamless Integration**: BYOK facilitates smooth integration between on-premises and cloud environments, enabling organizations to use the same encryption keys across different platforms. This flexibility enhances operational efficiency and reduces management overhead.

Vault offers several key lifecycle management solutions:

1. **Key Management Secrets Engine:** Enables centralized lifecycle management of cryptographic keys while allowing copies of the key material to be securely distributed to cloud key management service (KMS) providers for cryptographic operations.
2. **KMIP Secrets Engine:** Vault acts as a KMIP server, allowing services and applications to delegate cryptographic key management to Vault using a standardized protocol.

7 Transit and Transform secrets engines

> **Note**
>
> Transit and Transform engines, roles, access controls, and scaling are configured and managed by platform operators. See the Administration Guide: Transit and Transform management.

7.1 Overview

In today's digital landscape, protecting sensitive data is paramount. HashiCorp Vault offers two powerful solutions: the Transit engine, which focuses on encrypting and decrypting data without storage; and the Transform engine, which provides format preserving encryption and tokenization for sensitive data. Together, these engines help protect data both in transit and at rest. Implementing these secrets engines helps organizations:

- **Enhance Data Security**: Centralize encryption and tokenization processes to protect sensitive information.

- **Achieve Compliance**: Meet regulatory requirements such as PCI-DSS, GDPR, and HIPAA.
- **Improve Performance and Scalability**: Optimize cryptographic operations for high-throughput environments.
- **Simplify Key Management**: Streamline key creation, rotation, and destruction.
- **Maintain Data Usability**: Utilize data transformation techniques that preserve data formats, enabling seamless integration with existing systems.

7.1.1 Common use cases mapped to each encryption engine

Transit engine

- **Encryption-as-a-Service**: Ideal for encrypting data in transit or at rest without storing the data.
- **Use Cases**:
  - API payload encryption
  - Database field encryption
  - Securing sensitive configuration data

Transform engine

- **Format Preserving Encryption (FPE)**: Encrypt data while maintaining its original format.
- **Tokenization**: Replace sensitive data with tokens for secure handling.
- **Masking**: Obscure parts of data for privacy while retaining usability.
- **Use Cases**:
  - Protecting credit card numbers in retail (PCI-DSS compliance)
  - Tokenizing PII for analytics in healthcare (HIPAA compliance)
  - Masking data in customer service applications

7.2 Transit engine

Vault's Transit secrets engine provides cryptographic functions to encrypt, decrypt, sign, and verify data. It is designed to handle high-throughput encryption operations, making it ideal for use cases such as encrypting credit card information during transactions. By simplifying, abstracting, and offloading the encryption process, the Transit engine offers a scalable and secure solution for protecting sensitive data, allowing developers to focus on application logic.

7.2.1 Design considerations

1. Key Management and Rotation
    - Centralized key management: All encryption keys are centrally managed within Vault, providing a single point of control for key creation, rotation, and destruction.
    - Automatic key rotation: Configuring Vault to support automatic key rotation ensures that keys are regularly updated without disrupting ongoing operations, enhancing long-term security.
2. Performance and Scalability
    - Optimized for high throughput: The Transit engine is optimized to handle thousands of transactions per second (TPS) with minimal latency, making it ideal for high-volume applications.
    - Horizontal scalability: The Transit engine's operations are primarily reads. It does not store data, allowing for high horizontal scalability. Leverage this characteristic by implementing auto-scaling in cloud environments or by adding extra capacity to your infrastructure. This approach ensures that the Transit engine can dynamically adjust to varying loads, maintaining consistent performance as demand changes.
3. Security
    - TLS encryption: All communications between clients and the Transit engine are encrypted using TLS, safeguarding data in transit.

    > Note
    >
    > Specific TLS configuration depends on your Vault setup.

    - Access control: Strict access controls are enforced via Vault's policy framework, ensuring only authorized clients can perform operations using the Transit engine.
4. Encryption Algorithms
    - Algorithm diversity: The Transit engine supports a variety of encryption algorithms, including AES-256-GCM (default), ChaCha20-Poly1305, ED25519, and more.

7.2.2 Implementation

1. **Setup and Configuration**: For detailed implementation steps, please refer to the official Vault Transit secrets engine tutorial. This guide provides comprehensive, step-by-step instructions for setting up and using the Transit engine.

   Key implementation steps include:

   - enabling the Transit engine
   - creating encryptions keys
   - encrypting and decrypting data
   - configuring key rotation.

2. **Integration with Applications**: Integrate your application to call Vault's Transit API for encrypting credit card information before storing or transmitting data. Ensure your applications can handle the encrypted data formats.

3. **Error-handling and Logging**: Implement comprehensive error handling in your integration code to manage scenarios such as network issues, authentication failures, or Vault unavailability. Incorporate retry mechanisms with exponential backoff, and ensure that your application can gracefully handle exceptions without data loss. Enable and configure Vault's audit logging to track all Transit engine operations for security and compliance purposes.

7.2.3 Use cases covered

Use Case	Summary
Encrypting credit card information	In environments where sensitive data protection is crucial - such as retail, banking, or healthcare - it is essential to encrypt customer information. By using Vault's Transit engine, organizations can encrypt credit card data before storing it in a database or transmitting it across networks. This ensures that even if the data is intercepted or accessed without authorization, it remains protected.

Data flow overview

- **Encryption**: Before storing or transmitting, the application sends the plaintext credit card number to Vault's Transit engine to receive the cipher text.
- **Decryption**: When needed, the application sends the cipher text to Vault to retrieve the plaintext.

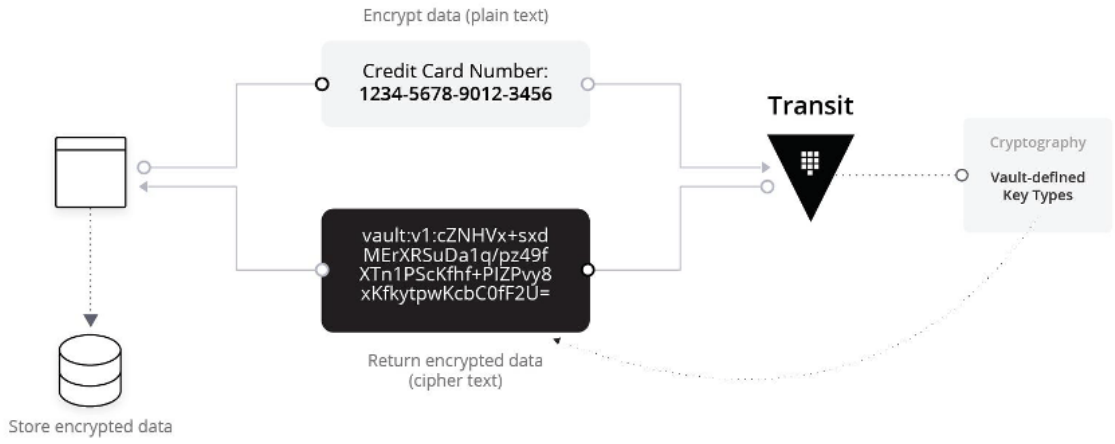


Figure 7: Data flow case

7.2.4 Best practices

- **Use Short-lived Tokens**: Implement short-lived tokens when interacting with Vault, including with the Transit engine. This practice significantly reduces the risk of compromise by limiting the lifespan of access credentials. Configure Vault's token policies to set appropriate maximum time-to-live (TTL) values for tokens. Integrating ephemeral tokens into your security framework significantly strengthens the security posture of your Vault deployment. This approach not only fortifies access control mechanisms, but also aligns with industry standards and regulatory requirements, ensuring compliance.
- **Regularly Rotate Keys**: Implement a key rotation policy that balances security with operational efficiency. Vault's Transit engine supports automatic key rotation, allowing you to schedule rotations at regular intervals or after a specific number of encryption operations.

  When implementing key rotation:

  - Use Vault's key versioning feature to maintain multiple versions of encryption keys. This is managed by Vault admins, not application developers.
  - Implement a rolling update strategy for rewrapping cipher text. Periodically rewrap encrypted data to use the latest key version. This can be done as a background process or during regular read operation.
  - Design your applications to handle potential temporary unavailability during key rotation operations. Implement retry logic for encryption and decryption operations to handle temporary failures.
  - Regularly test your key rotation and rewrapping procedures to verify that they function as expected and that the data remains accessible after a rotation.

7.3 Transform engine

Vault's Transform secrets engine is designed to protect sensitive data through mechanisms that preserve data formats, tokenize sensitive information, or apply data masking. This engine is particularly useful in scenarios where maintaining the usability of data — such as length or format — is critical while still ensuring that the data is secure.

The Transform engine ensures data security while preserving data usability through techniques like format preserving encryption (FPE) algorithms such as FF1 and FF3-1, secure tokenization, and customizable data masking.

Its flexibility makes it suitable for various use cases, from tokening personally identifiable information (PII) for compliance to applying FPE in financial services.

By leveraging the Transform engine, organizations can achieve:

- Enhanced data protection while preserving data usability,
- Compliance with various data privacy regulations,
- Improved performance for data transformation operations, and
- Simplified management of data protection processes across different use cases.

7.3.1 Implementation

1. **Enable the Transform Engine**: To begin using the Transform engine, it must first be enabled in Vault. This can be done using the following command:

   ```
   $ vault secrets enable transform
   ```

   Once enabled, you can configure the engine to meet your specific needs, including setting up transformations, defining role, and managing access controls.

2. **Defining Transformations**: Transformations are the core feature of the Transform engine. A transformation can be a process like FPE, tokenization, or data masking when applied to specific types of data.

7.3.2 Use cases covered

Use Case	Summary
FPE for structured data	Encrypting data such as SSNs or credit card numbers while preserving the original format, enabling seamless integration with existing systems that expect data in a specific format.

Use Case	Summary
Tokenization for compliance	Replacing sensitive data elements with tokens to ensure compliance with data privacy regulations, while maintaining data usability for business processes.
Masking for data privacy	Masking sensitive data fields to prevent unauthorized access while allowing partial visibility for operational use cases, such as displaying the last four digits of a credit card number.

7.3.3 Format preserving encryption

FPE allows you to encrypt data while preserving its original format. For example, a 16-digit credit card number can be encrypted but will still appear as a 16-digit number, which is essential for systems that require specific data formats.

To define a template and an FPE transformation, use the following command:

```
## Define a template for credit card numbers
$ vault write transform/template/ccn-template \
    type=regex \
    pattern='(\d{4})-(\d{4})-(\d{4})-(\d{4})' \
    alphabet=builtin/numeric
## Create an FPE transform using the template
$ vault write transform/transformation/fpe-ccn \
    type=fpe \
    template=ccn-template \
    tweak_sources=internal \
    allowed_roles=payments
```

In this example, the transformation parameter specifies the FPE method for encrypting credit card numbers, ensuring that the encrypted data retains its original structure.

To perform an FPE encoding of a sample credit card number, use the following command:

```
$ vault write transform/encode/payments \
    transformation=fpe-ccn \
    value=1111-2222-3333-4444

Key              Value
---              -----
encoded_value    9300-3376-4943-8903
```

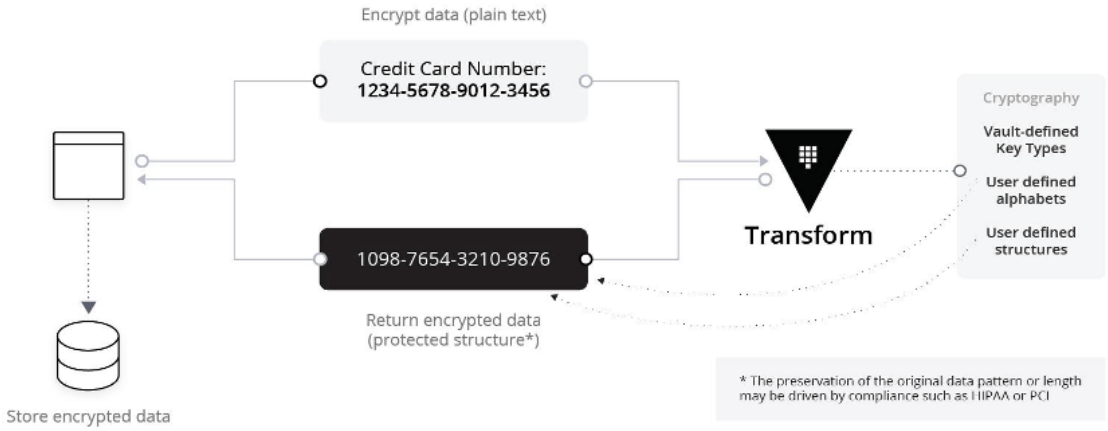


Figure 8: Credit card transformation

7.3.4 Tokenization

Tokenization replaces sensitive data with a token that has no exploitable meaning outside of Vault. This is particularly useful for securing PII, such as social security numbers (SSNs).

To encode a person's SSN using a tokenization transform, use the following command:

```
$ vault write transform/encode/payments value="123-45-6789" \
    transformation=us-ssn

Key              Value
---              -----
encoded_value    233-76-4943
```

The result is a token that represents the original SSN, but can be safely used without exposing the sensitive data.

Security considerations for tokenization

When implementing tokenization with the Transform engine, it is crucial to address specific security considerations to ensure the protection of sensitive data. Tokenization replaces sensitive data with tokens. If not properly secured, however, it can introduce vulnerabilities.

Key security considerations:

- **Token Storage Security**: Ensure that the tokenization storage backend - whether using Vault's integrated storage or an external SQL database - is securely configured. Implement strong access controls, encryption at rest, and regular security audits to protect token-to-value mappings from unauthorized access.
- **Access Controls**: Use Vault's policy framework to enforce strict access controls on tokenization operations. Define clear roles and permissions to ensure that only authorized users or services can perform encoding and decoding operations.
- **Data Minimization**: Limit the amount of sensitive data processed and stored. Only tokenize data that is necessary for your application's functionality to reduce exposure.
- **Audit Logging**: Enable Vault's audit logging to monitor all tokenization activities. Regularly review audit logs to detect and respond to any unauthorized access attempts or suspicious activities.
- **Compliance Alignment**: Ensure that your tokenization implementation aligns with relevant regulatory requirements such as PCI-DSS, GDPR, or HIPAA. Properly securing tokenization processes aids in maintaining compliance and avoiding potential penalties.

For detailed information on security considerations for tokenization, please refer to Security Considerations for Tokenization.

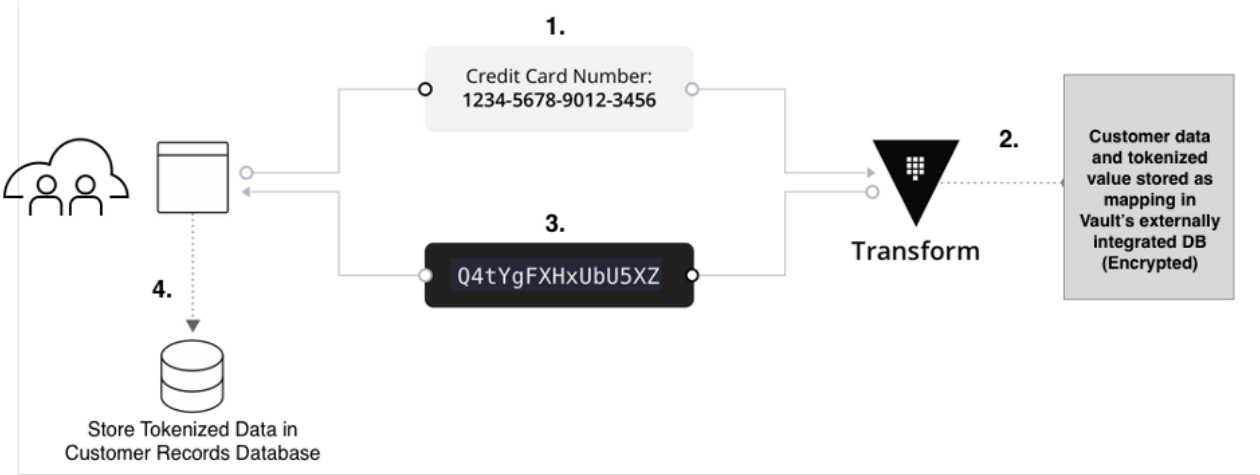


Figure 9: Credit card tokenization

7.3.5 Data Masking

Data masking is used to partially obfuscate data, allowing for limited visibility. It is intended to hide sensitive data from those who do not have a clearance to see them, while retaining data format options for operational purposes. To implement data masking of a phone number, use the following command:

```
$ vault write transform/encode/payments value="+1 123-345-5678" \
    transformation=phone-number

Key              Value
---              -----
encoded_value    +1 ###-###-####
```

This will output a masked version of the phone number containing "#" as the masking character as defined in the transformation rule.

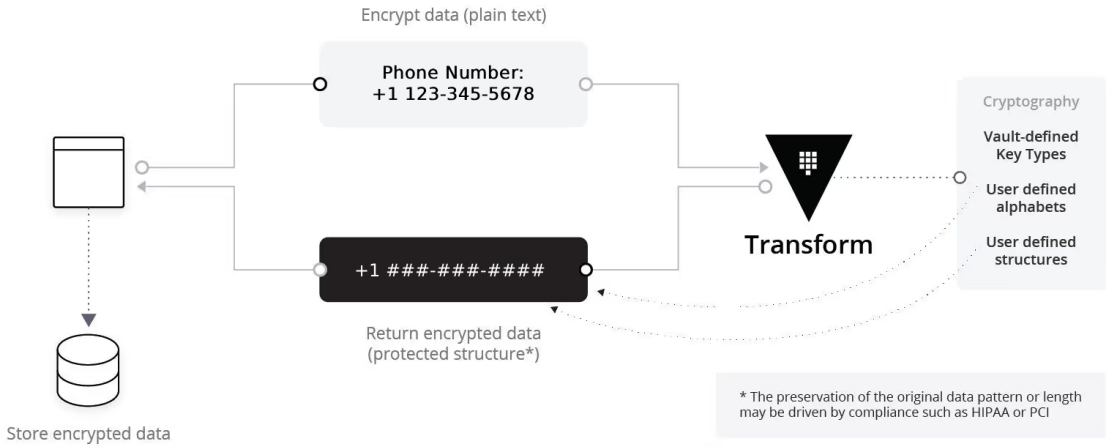


Figure 10: Credit card masking

7.3.6 Managing roles and access

Roles are essential for controlling who can access and perform operations within the Transform engine. Each role should be tied to specific transformations and operations, ensuring that clients have only the permissions they need.

- **Creating a Role**: Define roles that specify which transformations users or service can access.

```
$ vault write transform/role/payments \
    transformations=card-number,uk-passport,us-ssn
```

This role allows access to the card-number, uk-passport, and us-ssn transformations.

- **Assigning Permissions**: Use Vault policies to assign specific permissions to roles, such as the ability to read, write, or manage transformations.

```
path "transform/encode/payments" {
    capabilities = ["create", "update"]
}

path "transform/decode/payments" {
    capabilities = ["create", "update"]
}
```

7.3.7 Implementation best practices

Integrating the Transform engine effectively with your existing databases and applications is essential for a successful deployment. Start by identifying the applications or services that will interact with the engine for encryption, tokenization, or data masking. Update your application logic to utilize the Transform engine's APIs whenever sensitive data needs processing, ensuring your applications can smoothly handle the transformed data formats.

Implement HTTP connection pooling to optimize performance during Transform operations, This minimizes the overhead of establishing new connections for each API call by reusing HTTP connections, thereby reducing latency from repeated TCP handshakes and SSL/TLS negotiations. Configure the maximum total connections and maximum connections per route to match your application's concurrency needs. Set appropriate connection and socket timeouts to prevent hanging connections. Incorporate error handling strategies for network issues, including retries with backoff mechanisms.

When scaling tokenization, consider configuring external SQL stores to manage tokenization mappings. Set up an external SQL database, such as MySQL, PostgreSQL, or Microsoft SQL Server to store the relationships between plaintext values and their corresponding tokens. Optimize your database performance to handle the frequent read and write operations involved in tokenization. Ensure high availability by implementing replication and clustering for your database, providing fault tolerance and minimizing the risk of disruptions in tokenization services due to database failures.

7.3.8 Performance characteristics

- **Optimizing Encryption Operations and Batch Processing**: To ensure optimal performance of format preserving encryption (FPE) and other Transform operations in high-volume environments, it is crucial to establish and monitor key performance metrics. Begin by setting baseline response times for individual Transform operations and tracking operations per second. Regularly monitor these baselines for any significant deviations, which could indicate any performance issues. Additionally, monitor CPU utilization across Vault nodes, tracking both average and peak levels. To process large datasets, implement batch processing for bulk Transform operations. This approach can significantly reduce overhead and improve overall throughput compared to processing individual items, making it an effective strategy for handling high volumes of data. Batching is particularly useful in environments where FPE is applied to large datasets, such as batch processing of customer data. Additionally, con-

sider implementing caching strategies to further optimize performance. Caching frequently encrypted values can reduce the load on Vault, especially when the same data requires repeated encryption operations. This is particularly relevant in scenarios where certain data fields, such as SSNs, are encrypted multiple times across different applications.

- **High-volume Tokenization Management**: Tokenization replaces sensitive data elements with tokens that can be used in place of the original data for business processes. To manage tokenization at scale, ensure that the Transform engine is configured to handle high throughput. This involves optimizing the database backend, if an external SQL database is used to persist the tokens. Regular performance tuning, such as indexing the database and optimizing query execution plans, can significantly enhance the speed and efficiency of tokenization operations. Additionally, use monitoring tools to track the performance of the tokenization engine to adjust as needed for maintaining optimal throughput.
- **Latency Reduction**: In environments with strict performance requirements, such as real-time processing of financial transactions, minimizing latency is critical. Deploying Vault clusters close to the data sources and clients can reduce the time it takes for tokenization requests to be processed. Implementing load balancers to distribute tokenization requests evenly across multiple Vault nodes also helps in maintaining consistent performance under high load conditions.

8 Transit and Transform examples

While the previous use cases cover common applications of the Transit and Transform engines, the following examples demonstrate the breadth and depth of scenarios where these powerful tools can be applied. These use cases span various industries and compliance requirements, highlighting the versatility and effectiveness of Vault's data protection capabilities.

8.1 Database column-level encryption with the Transit engine

To implement database column-level encryption using HashiCorp Vault's Transit engine, we recommend following the Vault Transit Rewrap tutorial. This hands-on tutorial demonstrates:

- **Enabling the Transit Engine**: Set up and configure the Transit secrets engine for encryption operations.

- **Enabling Data Before Storage**: Use the Transit API to encrypt sensitive data before inserting it into your database.
- **Decrypting Data After Retrieval**: Decrypt data retrieved from the database securely using the Transit engine.
- **Performing Key Rotation and Data Rewrapping**: Regularly rotate encryption keys and rewrap existing data to maintain strong security practices.

8.2 Securing sensitive healthcare data with the Transform engine

Healthcare organizations often need to perform analytics on sensitive patient data while maintaining compliance with regulations like HIPAA. The Transform engine's format preserving encryption (FPE) capabilities allow for the encryption of sensitive data in a way that preserves its format, enabling analytics while protecting patient privacy.

1. Enable the Transform Engine

```
vault secrets enable -path=healthcare-transform transform
```

2. Configure Templates

```
## Create the SSN template
$ vault write healthcare-transform/template/ssn \
    type=regex \
    pattern='(\d{3})-(\d{2})-(\d{4})' \
    alphabet=builtin/numeric

## Create MRN template (10-digit format)
$ vault write healthcare-transform/template/mrn \
    type=regex \
    pattern='(\d{10})' \
    alphabet=builtin/numeric
```

3. Create Transformations

```
## Configure SSN transformation
$ vault write healthcare-transform/transformation/ssn-fpe \
    type=fpe \
    template=ssn \
    tweak_source=internal \
    allowed_roles=healthcare-analytics

## Configure MRN transformation
$ vault write healthcare-transform/transformation/mrn-fpe \
    type=fpe \
    template=mrn \
    tweak_source=internal \
    allowed_roles=healthcare-analytics
```

4. Create a Role

```
## Create a role with access to both transformations
$ vault write healthcare-transform/role/healthcare-analytics \
    transformations=ssn-fpe,mrn-fpe
```

5. Transform Data

```
## Encrypt SSN
$ vault write healthcare-transform/encode/healthcare-analytics \
    transformation=ssn-fpe \
    value="123-45-6789"

## Encrypt MRN
$ vault write healthcare-transform/encode/healthcare-analytics \
    transformation=mrn-fpe \
    value="1234567890"

## Decrypt transformed SSN (when necessary)
$ vault write healthcare-transform/decode/healthcare-analytics \
    transformation=ssn-fpe \
    value="<encoded-value-from-previous-step>"
```

6. Configure Access Control

```
## Create a policy for the Transform operation
$ vault policy write transform-healthcare-policy - <<EOF
path "healthcare-transform/encode/healthcare-analytics" {
    capabilities = ["create", "update"]
}

path "healthcare-transform/decode/healthcare-analytics" {
    capabilities = ["create", "update"]
}
EOF
```

Example outputs

```
## Sample SSN transformation
$ vault write healthcare-transform/encode/healthcare-analytics \
    transformation=ssn-fpe \
    value="123-45-6789"

Key              Value
---              -----
encoded_value    891-24-3567

## Sample MRN transformation
$ vault write healthcare-transform/encode/healthcare-analytics \
    transformation=mrn-fpe \
    value="1234567890"

Key              Value
---              -----
encoded_value    8915673421
```

You can monitor the Transform FPE operations through Vault's audit logs by tracking request to the configured mount path (`healthcare-transform/`). For comprehensive guidance on configuring audit logging, refer to the Vault validated design guidance.

Key metrics to monitor:

- Success and failure rates of transformation operations.
- Access patterns to specific transformations.
- Response times for FPE operations.

8.3 Implementing GDPR compliance using Transform tokenization

The General Data Protection Regulation (GDPR) requires organizations to implement appropriate measures to protect personal data. Vault's Transform engine provides tokenization capabilities that can replace personally identifiable information (PII) with secure tokens while maintaining data usability. This section demonstrates how to implement tokenization for GDPR compliance.

1. Enable the Transform Engine

```
$ vault secrets enable -path=gdpr-transform transform
```

2. Create Templates

```
## Create template for names
$ vault write gdpr-transform/template/name \
    type=regex \
    pattern='([A-Za-z]+)( [A-Za-z]+)*' \
    alphabet=builtin/alphalower

## Create template for email addresses
$ vault write gdpr-transform/template/email \
    type=regex \
    pattern='([A-Za-z0-9._%+-]+)@([A-Za-z0-9.-]+\.[A-Za-z]{2,})' \
    alphabet=builtin/alphanumeric
```

3. Configure Transformations

```
## Create transformation for names
$ vault write gdpr-transform/transformation/name-tokenization \
    type=tokenization \
    template=name \
    allowed_roles=gdpr-admin \
    max_ttl="720h"

## Create transformation for email addresses
$ vault write gdpr-transform/transformation/email-tokenization \
    type=tokenization \
    template=email \
    allowed_roles=gdpr-admin \
    max_ttl="720h"
```

4. Create a Role

```
## Create a role with access to both transformations
$ vault write gdpr-transform/role/gdpr-admin \
    transformations=name-tokenization,email-tokenization
```

5. Tokenize PII Data

```
## Tokenize a name
$ vault write gdpr-transform/encode/gdpr-admin \
    transformation=name-tokenization \
    value="Matt Black"

## Tokenize an email address
$ vault write gdpr-transform/encode/gdpr-admin \
    transformation=email-tokenization \
    value="matt.black@aol.com"
```

6. Retrieve Original Data

```
## Retrieve original name when needed
$ vault write gdpr-transform/decode/gdpr-admin \
    transformation=name-tokenization \
    value="<tokenized-name-value>"

## Retrieve original email when needed
$ vault write gdpr-transform/decode/gdpr-admin \
    transformation=email-tokenization \
    value="<tokenized-email-value>"
```

8.3.1 Using curl with Vault's API

```
## Set your Vault token and address
$ export VAULT_TOKEN="your-token"
$ export VAULT_ADDR="https://vault.example.com:8200"

## Tokenize data
$ curl --header "X-Vault-Token: $VAULT_TOKEN" \
    --request POST \
    --data '{"transformation": "name-tokenization", "value":"John Doe"}' \
    $VAULT_ADDR/v1/gdpr-transform/encode/gdpr-admin

## Retrieve original data
$ curl --header "X-Vault-Token: $VAULT_TOKEN" \
    --request POST \
    --data '{"transformation": "name-tokenization", "value": "<tokenized-value>"}' \
    $VAULT_ADDR/v1/gdpr-transform/decode/gdpr-admin
```

8.3.2 Example Outputs

```
## Tokenizing a name
$ vault write gdpr-transform/encode/gdpr-admin \
    transformation=name-tokenization \
    value="Matt Black"

Key              Value
---              -----
encoded_value    tok_Gl8qH4hJns9QR3U

## Retrieving original name
$ vault write gdpr-transform/decode/gdpr-admin \
    transformation=name-tokenization \
    value="tok_Gl8qH4hJns9QR3U"

Key              Value
---              -----
decoded_value    Matt Black
```

8.3.3 Benefits

- **GDPR Compliance**: Helps meet GDPR requirements for data protection and tokenization through secure tokenization of personal data and comprehensive audit capabilities.
- **Data Protection**: Enhances security by storing tokens instead of original PII in databases, with options for external token storage.
- **Data Usability**: Enables data processing on tokenized data without exposing personal information, maintaining functionality while protecting privacy.
- **Access Control**: Provides granular control through separate roles for encode and decode operations, allowing organizations to limit access based on specific business needs.

9 RHEL local account password rotation

> **Note**
>
> This guide has software version dependencies. Refer to the OS secrets engine documentation for the current requirements.

Treat local account password rotation as a focused operating pattern for brownfield systems, not as a replacement for centralized identity. We recommend using it for a narrow class of local accounts outside centralized identity systems. That class includes break-glass accounts, privileged management accounts, and other long-lived local credentials that are difficult to eliminate but risky to leave outside formal governance.

This capability is most valuable when you already run Vault as a shared service and need to reduce the security and compliance burden of manually rotated passwords. The goal is not to make local accounts a preferred pattern. The goal is to make the remaining local-account footprint smaller, safer, and easier to govern while you continue moving toward stronger identity-based patterns elsewhere.

This guide covers the OS secrets engine on Vault Enterprise, connecting to Red Hat Enterprise Linux (RHEL) hosts over direct SSH. It does not cover Windows or WinRM targets, Vault Community Edition, or other secrets engines.

9.1 Start with the right use cases

We recommend adopting local account password rotation only for accounts that meet all of the following criteria:

- The account already exists on the target host and remains long-lived.
- The current operating model cannot realistically replace the account with a centralized identity system.
- The account represents meaningful operational or security risk if its password remains static.
- The account is important enough to justify controlled onboarding, rotation, and recovery procedures.

Typical examples include:

- break-glass local accounts retained for loss-of-directory or loss-of-network scenarios
- privileged accounts with `sudo` access or management accounts used to repair host configuration
- local service credentials on legacy systems that cannot yet move to a different pattern

We do **not** recommend leading with this feature for standard human access, broad privileged access management, or general account lifecycle management. If a centralized directory already solves

the identity problem for the target users, keep using it. If a use case can move to short-lived credentials instead, prioritize that path first.

9.2 Treat this as a standardization pattern, not a one-off feature

The strongest HVD pattern for this capability is to standardize how platform teams govern exceptions. In practice, that means using Vault to make local account management more predictable across many systems instead of solving each host independently.

We recommend the following operating model:

1. Define a small set of approved account classes for onboarding.
2. Standardize how teams represent hosts, name accounts, and grant onboarding authority.
3. Apply the same rotation policies, audit expectations, and recovery procedures across each account class.
4. Expand gradually only after the platform team has validated that rotation, recovery, and operational ownership are working well.

This feature reduces operational variance only when teams use it consistently. If each team onboards hosts differently, handles failures differently, and stores recovered credentials differently, you reproduce the same inconsistency that made the local-account footprint risky in the first place.

9.3 Prefer the least-privileged rotation model

We recommend **self-managed (rootless) rotation as the default pattern** whenever the target account can change its own password under the host's approved controls. This keeps the lifecycle of the target account isolated and avoids introducing a second privileged credential purely to support rotation.

Use a **parent-managed** model only when the account cannot rotate itself or when operational constraints require a separate privileged helper account. In that case:

- Create a dedicated parent account for rotation rather than reusing a broadly privileged administrator account.
- Scope the parent account to the minimum rights needed to change the target password.
- Constrain `sudo` permissions to the exact binary and workflow required for rotation.
- Document which target accounts depend on that parent account for continued rotation.

The important design principle is to avoid turning a password-rotation feature into a new standing administrator pattern. If parent-managed rotation becomes the default for every account, the blast radius of the helper credential becomes the real problem you now manage.

9.4 Onboard conservatively

We recommend a deliberate onboarding sequence instead of bulk importing accounts and scheduling rotation in the same change window. Start with host onboarding, validate connectivity and trust, then onboard accounts in a controlled cohort.

Set connection rules when you onboard the host. Standardize the host address, port, and host key material, and make Vault verify the first real connection before the platform accepts an account into service. Avoid deferring that first verification until the first scheduled rotation unless the platform team has accepted the risk and documented the exception.

Two practices matter here:

- **Do not assume import equals readiness.** Follow account onboarding with a deliberate validation step and then a manual rotation before you enable scheduled rotation.
- **Preserve reversibility.** During early deployment, make sure the platform team can retrieve the current credential, verify that rotation succeeded, and offboard the account cleanly if the target system is not yet ready.

This capability fits phased adoption better than a big-bang deployment. We recommend beginning with a small set of non-critical Linux hosts, validating the end-to-end workflow, and then expanding to higher-sensitivity accounts once the run book is stable.

9.5 Make host trust explicit

We recommend treating host key verification as a required part of the operating pattern, not an optional hardening step. Wherever possible, pre-populate the host key during onboarding and verify it before the account enters rotation.

Trust on First Use (TOFU) can help bootstrap environments that do not yet have a better distribution path for host keys, but use it as a transitional mechanism rather than the long-term standard. If you rely on TOFU broadly, you weaken one of the most important controls protecting the rotation workflow from connecting to the wrong system.

A good enterprise pattern is:

- Use explicit host key management for stable production hosts.
- Define how teams handle host key rotation before onboarding at scale.
- Require a documented approval path for any TOFU-based onboarding in higher-sensitivity environments.

9.6 Design around the real platform dependencies

Local account password rotation succeeds or fails based on platform conditions outside Vault. Before you scale this pattern, we recommend validating the dependencies that most often create operational friction:

- A direct SSH path from Vault to the target host.
- Plugin registration and version alignment as part of the Vault deployment plan.
- The required binaries present in the host path.
- Pseudo-terminal (PTY) support when using self-managed rotation.
- `sudo` configuration scoped to the exact commands required for parent-managed rotation.
- An explicit password policy so rotated credentials meet complexity requirements consistently across hosts, rather than relying on host or plugin defaults.

This is one reason to keep the first deployment narrow. The feature uses a specific rotation model. If your environment depends on bastion hops, outbound proxies, or inconsistent host hardening standards, fix those dependencies first or explicitly exclude those hosts from the initial operating model.

9.7 Separate rotation control from credential consumption

We recommend keeping the number of actors that can read rotated passwords as small as possible. The platform value of this feature comes from consistent rotation and recovery, not from making local account passwords easier to distribute.

Where applications or automation need the credential, prefer a controlled consumption pattern that already aligns with the broader Vault operating model. For legacy consumers, Vault Agent is often the best bridge: it is a credential consumption and delivery pattern, not a rotation mechanism, so it

reduces custom integration work while letting the platform team standardize how rotated secrets reach existing systems.

In practice, this means:

- Separate policies for onboarding and rotation from policies that read current credentials.
- Reserve direct credential retrieval for controlled operational use cases, recovery workflows, and approved consumers.
- Avoid normalizing informal password reads by operators when the real need is better automation or templating.

If teams still depend on humans copying passwords out of Vault as a steady-state workflow, the design is not yet mature enough to scale.

9.8 Use manual rotation to prove readiness, then automate

We recommend using **manual rotation as the readiness gate** and **scheduled rotation as the steady state**.

Manual rotation is the right place to validate:

- The chosen rotation model.
- Connectivity and execution behavior on the host.
- The account's post-rotation usability.
- The team's recovery steps if something goes wrong.

Once that works consistently, enable automated rotation with an interval that reflects the sensitivity of the account and the operational tolerance of the workload. Choose rotation cadence with the dependent systems and operating team in mind. Tie it to the time required for dependent systems to absorb a password change and for the platform team to respond when a host drifts out of sync.

9.9 Make offboarding a standard workflow

We recommend treating offboarding as part of the managed lifecycle rather than as cleanup after teams finish the real work. When an account leaves managed rotation, turn off automated rotation first, retrieve the current credential for the next approved owner, and then remove the managed account entry. Delete a host entry only after the platform team has confirmed that no remaining managed accounts depend on it.

This sequence keeps ownership clear at the point where drift and access loss are most likely. It also prevents teams from deleting Vault state before they have captured the final known-good credential or confirmed that another workflow now governs the account.

9.10 Plan for failure before you expand

This feature manages credentials on a remote system, so recovery planning is part of the design, not an afterthought. We recommend documenting a manual recovery workflow before scaling beyond the pilot:

- How to identify whether Vault or the host has the authoritative password.
- How to use version history and audit data to determine the correct recovery point.
- When to restore a historical version versus when to perform a controlled reset.
- How to offboard an account without losing access or audit continuity.

The key principle is straightforward: do not expand the scope of managed accounts until the platform team can recover from split-brain scenarios confidently and repeatably.

9.11 Roll out by account class, not by host count

We recommend sequencing adoption by operational similarity instead of trying to rotate every eligible account at once. A practical deployment sequence looks like this:

1. **Pilot:** a small supported Red Hat Enterprise Linux (RHEL) cohort with low operational variance.
2. **Break-glass accounts:** accounts where governance and audit readiness matter most.
3. **Management accounts:** repair accounts or accounts with `sudo` access and clearer owner responsibilities.
4. **Broader legacy coverage:** only after rotation, recovery, and policy boundaries are working consistently.

This order helps the platform team learn where the real failure modes are without spreading the feature faster than the organization can support.

9.12 Reference materials

Use the product documentation for feature-specific configuration and API behavior, and use this HVD guidance to shape the operating model around that implementation:

- Vault Agent
- OS secrets engine
- Vault plugins
- SSH secrets engine
- Automate Linux password rotation with Vault

10 Vault Agent templates

Vault Agent templates offer a seamless way to integrate secrets management into your applications both new and old/legacy. For legacy (existing) applications, Vault agent brings the benefit of modern secrets management bypassing the need for application upgrades or modifications. These agents authenticate with Vault, retrieve secrets, and inject them where your application expects, either as an environment variable or file, facilitating secret consumption and refreshment in sync with the application's lifecycle.

10.1 Understanding Vault Agent templates

Vault Agent templates bridge the gap between the secret's format in Vault and the application's expectations. For instance, consider an application that uses a credentials file like this:

```
Password=passwordxyz,
Username=userxyz
```

Vault's API responses look like this:

```
"Password": "passwordxyz",
"Username": "userxyz"
```

Agent templates allow you to reformat this data, ensuring it aligns with the application's required input format.

10.2 Getting Started with Vault Agent Templates

> **Note**
>
> Vault Agent leverages the same templating engine used by Hashicorp Consul and other HashiCorp products. For more details on the templating engine, see the consul-template public GitHub repository.

To demonstrate the components of templates, we will create a template that renders a file in a format that another application expects, populated with secret information from Vault. Given an existing secret in Vault like this:

```
$ vault read database-app1/creds/pg-ro

Key                Value
---                -----
lease_id           database-app1/creds/pg-ro/2f6a614c-4aa2-7b19-24b9-ad944a8d4d
lease_duration     1h
lease_renewable    true
password           FSREZ1S0kFsZtLat-y94
username           v-vaultuser-e2978cd0-ugp7iqI2hdlff5hfjylJ-1602537260
```

We would expect the final rendering of the secret within the destination to be:

```
DBUser:v-vaultuser-e2978cd0-ugp7iqI2hdlff5hfjylJ-1602537260
DBPassword:FSREZ1S0kFsZtLat-y94
```

10.2.1 Template language overview

Templates are defined using Go's templating format. First, we define a `template_config` block in our agent configuration to control template interactions. This block, set globally, specifies error handling and render intervals for static secrets:

```
template_config {
  exit_on_retry_failure = false
  static_secret_render_interval = "5m"
}
```

The above serves as an example of how the configuration can be changed and is based on the original default value.

> **Note**
>
> The rendering frequency for both static and dynamic secrets is specified based on whether it is a leased and/or renewable secret. Each combination has its own default configuration for when the rendering will happen. In most cases, the default setting should be sufficient. Otherwise, those configurations can be re-configured as indicated here.

Next, we define how to format the rendered secret within a `template` stanza. This stanza is specifically for manipulating a secret pulled from the Vault server and formatting in a way the consuming application expects:

```
template {
  source      = "/tmp/agent/template.ctmpl"
  destination = "/tmp/agent/render.txt"
}
```

This configuration specifies the source template file and the destination file for the rendered output.

> **Tip**
>
> It is possible to render multiple secrets to a specific destination file within the same template. This is applicable in scenarios where you have a single application config file containing multiple secrets. Likewise, it is also possible to define multiple template stanzas within the same agent configuration file secrets for the creation of multiple destination files.

The source value in the template stanza references a templating file (.ctmpl) that specifies the Vault secret to retrieve and its formatting. The type of secret dictates the function used to declare it in the template.

The `.ctmpl` file should resemble the following:

```
{{ with secret "database-app1/creds/pg-ro" }}
DBUser:{{ .Data.username }}
DBPassword:{{ .Data.password }}
{{ end }}
```

The template's first line declares the secret function, essential for processing most Vault secrets, excluding PKI certificates (we'll cover this exception later). The rest is written normally in the format of the destination file, using curly braces to break out into rendering the secret as needed. Any

characters that fall outside of the curly braces will be taken literally. When a secret is pulled from Vault, it is returned as a JSON-based payload.

> **Tip**
>
> If there are errors in the templating engine, directly make an API call to Vault and examine the JSON payload to ensure that the template's configuration aligns with the response received from Vault.

A full list of parameters for the template block can be referenced here.

For simpler scenarios where maintaining a separate template file is excessive, embed the template directly using the `contents` parameter in place of `source`:

```
template {
    contents    ="{{ with secret \"database-app1/creds/pg-ro\" }}DBUser:{{ .Data.
        username }}\nDBPassword:{{ .Data.password }}{{ end }}"
    destination ="/tmp/agent/render.txt"
}
```

10.2.2 Environment variables

For environment variable injection, use `env_template` stanzas. In this case, the name of the block (the string following `env_template`) will become the rendered environment variable. Because of this, no `destination` parameter is needed within `env_template` stanzas:

```
env_template "DB_USER" {
  contents = "{{ with secret \"database-app1/creds/pg-ro\" }}{{ .Data.username }}{{
      end }}"
}

env_template "DB_PASSWORD" {
  contents = "{{ with secret \"database-app1/creds/pg-ro\" }}{{ .Data.password }}{{
      end }}"
}
```

The full list of parameters for `env_template` can be found here.

10.2.3 Application memory

Some applications retrieve secrets directly from memory, not from files or environment variables. In these cases, it is recommended to use Vault Agent's Process Supervisor Mode. In this mode, Vault Agent will inject secrets referenced in the `env_template` configuration blocks as environment variables into the child process specified in the exec block.

> **Note**
>
> It is necessary for the target application to be run as a subprocess of Vault Agent (via exec) in order for Vault Agent to inject secrets into the target application's environment variable. In many ways, Vault Agent will mirror the child process. Standard input and output streams (stdin / stdout / stderr) are all forwarded to the child process. Additionally, Vault Agent will exit when the child process exits on its own with the same exit code.

Vault Agent's Process Supervisor mode is used when you define `env_template` and a separate `exec` block to perform the additional necessary command. This block is executed after all `env_template` have been rendered and will be executed again during any re-rendering of the `env_template` (e.g., secret rotation and update) :

```
env_template "DB_USER" {
  contents          = "{{ with secret \"database-app1/creds/pg-ro\" }}{{ .Data.
      username }}{{ end }}"
  error_on_missing_key = true
}

env_template "DB_PASSWORD" {
  contents          = "{{ with secret \"database-app1/creds/pg-ro\" }}{{ .Data.
      password }}{{ end }}"
  error_on_missing_key = true
}

exec {
  command                   = ["./my-app", "arg1", "arg2"]
  restart_on_secret_changes = "always"
  restart_stop_signal       = "SIGTERM"
}
```

Tip

It is recommended to set `error_on_missing_key = true`so that the Vault Agent will log an error when accessing a secret that does not exist. The error message will help during the debugging of the Vault Agent configuration. When `error_on_missing_key` is not specified or set to false (the default) and the key to render is not in the secret's response, the templating engine will ignore it (or render "value") and continue with its rendering.

10.2.4 PKI certificates

In the examples so far, we've used the `secret` function within the `template` stanza for retrieving secrets, However, PKI certificates require additional considerations, due to their inherent lifecycles and expiration details. Vault Agent provides a specific `pkiCert` template function to aid in the management of both the token/lease lifecycle and the certificate's expiration. This changes the behavior of when the agent refreshes the certificate, and how the agent will behave when it is restarted (e.g., whether to fetch an updated certificate or not based on the expiration of the certificate itself).

Here are some behavior differences to be aware of when working with PKI certificates:

- If a certificate is configured to be rendered using the `pkiCert` template function, Vault Agent will perform a check during agent startup to detect if there is an existing certificate. If none has been previously rendered or the current rendered certificate has expired, Vault Agent will fetch a new certificate. This same check is performed during the agent auto-auth re-authentication.
- If a certificate is rendered using the `secret` template function, Vault Agent will fetch a new certificate on agent startup, **even if previously rendered certificates are still valid**. This is also true during agent auto-auth re-authentication.
- In addition, if `generate_lease` is unset or set to **`false`**, the agent will use the certificate's `validTo` field to decide when to rotate the certificate. If `generate_lease` is set to **`true`**, the agent will follow the non-renewable, leased secret rules.

As a result of the above behavior with `secret`,it is therefore recommended to use `pkiCert` to render the PKI certificates so as to avoid creating unnecessary certificates even when the existing has not expired.

Common use cases for the `pkiCert` function to render TLS certificates

To generate a new certificate and create a bundle with the key, certificate, and CA, use:

```
{{ with pkiCert "pki/issue/my-domain-dot-com" "common_name=foo.example.com" }}
{{ .Data.Key }}
{{ .Data.Cert }}
{{ .Data.CA }}
{{ end }}
```

To fetch only the issuing CA for this mount, use:

```
{{- with secret "pki/cert/ca" -}}
{{ .Data.certificate }}
{{- end -}}
```

> Tip
>
> In the above example, `pki/cert/ca_chain` can be used instead to fetch the full CA chain.

10.3 Integration Strategies for Applications

Minimizing Disruption

As with any change in an environment, configuration changes on Vault Agent such as templates should be tested in a development environment and rolled up to QA/Prod in stages to minimize the chance of an outage. Error handling while reviewing in a dev environment is key to shield against possible issues such as (but not limited to):

- Agent process suddenly being stopped
- Application being stopped
- Reboots / Power outages
- Vault server unavailability
- File system permissions

10.4 Best practices for using HashiCorp Vault Agent templates in applications

10.4.1 Always use least-privilege access

Ensuring least privilege access is always best practice. As Vault Agent is a client of Vault server, consider the authentication method used by auto-auth and the ACL policy that is assigned upon successful authentication.

10.4.2 Error Handling

The base `template_config` stanza will be the first location to add some error handling, the only parameters available within this block are:

- `exit_on_retry_failure` (bool: **`false`**) - This option configures Vault Agent to exit after it has exhausted its number of template retry attempts due to failures.
- `static_secret_render_interval` (string or integer: `5m`) - If specified, configures how often Vault Agent Template should render non-leased secrets such as `kvv2`. This setting will not change how often Vault Agent templating renders leased secrets. Uses duration format strings.
- `exit_on_retry_failure` as a global parameter is a general catch-all which will go on a retry loop. Error handling can also be added within the template stanza while the secret is being processed to exit. An example of this is using `error_on_missing_key` within the `template` stanza.
- `error_on_missing_key` (bool: **`false`**) - Exit with an error when accessing a struct or map field/key that does not exist. The default behavior will print &`lt;no value`> when accessing a field that does not exist. It is highly recommended you set this to **`true`**.

Note

If `error_on_missing_key` is set to **`false`** or missing from the stanza, the templating will still attempt to render. If the expected payload is missing, the resulting destination will be blank.

10.5 Operational Best Practices

10.5.1 Monitoring and Logging

If the Vault Agent process is stopped or dies, applications will continue functioning with previously rendered secrets. However, it can still lead to issues, particularly if dynamic secrets with shorter life cycles are being used. Because of this, monitoring the health of Vault Agent and alerting on failures is crucial, especially as the number of applications using Vault Agent grows over time.

To address the above, Vault Agent has a telemetry feature that can be configured using the telemetry stanza to offer operational insights. As the telemetry stanza is exactly the same as those used to configure Vault Server telemetry, it is possible to leverage the same telemetry sinks used in your Vault Server deployment such as Prometheus, StatsD, or InfluxDB.

Vault Agent Telemetry provides the following runtime metrics about its performance, the auto-auth and the cache status:

For active monitoring of the operation logs, logging can be configured during the starting of the agent or specified in the configuration file. Vault Agent supports various verbosity levels such as trace, debug, info, warn, and error (in order of descending detail). It is recommended to keep the default verbosity level ("info") unless there are specific needs, such as during debugging, where you might need the debug or trace verbosity level for more detailed logs.

Just as with telemetry, it is recommended to integrate Vault Agent logging with your enterprise logging solution (if applicable) so the Vault logs can be consolidated and made visible through a centralized logging platform. Depending on your enterprise logging solution, it is also recommended to set up alerts for any logs indicating warnings or errors.

10.5.2 Common Issues and Solutions

- Customers sometimes encounter challenges when generating a Vault Agent's configuration file. A practical starting point is to use the agent binary's `generate-config` flag, which can help you create a basic configuration file. However, this initial setup typically needs fine-tuning before it is suitable for use in production, especially regarding things like authentication configuration or integrating different secret engines.
- If you find you need help debugging your template, you can monitor the agent log by setting the `log-level` to `trace`. This log level can provide useful data if you are experiencing issues initiating agent process, or if your secret is not being rendered correctly.

10.6 Additional Resources

- Links to relevant existing support articles or other HashiCorp documentation (optional section: integrate links throughout content wherever possible)
  - https://developer.hashicorp.com/vault/docs/agent-and-proxy/agent/template
  - https://developer.hashicorp.com/vault/docs/agent-and-proxy/agent/process-supervisor#env_template
  - https://github.com/hashicorp/consul-template/blob/main/docs/templating-language.md
  - https://developer.hashicorp.com/vault/docs/agent-and-proxy/agent
  - https://developer.hashicorp.com/vault/api-docs/secret/pki

11 Consume secrets in Kubernetes

HashiCorp Vault has an operator, sidecar container , and daemonset options available to provide flexible consumption of secrets in Kubernetes environments. Each of these provide different features, and advantages and disadvantages depending on how the Platform Team manages Kubernetes within the organization.

11.1 Vault Secrets Operator (VSO)

The Vault Secrets Operator manages secrets on Kubernetes using Vault. The primary utility of the Operator lies in its ability to facilitate the consumption of secrets as native Kubernetes secrets. The Operator renders secrets natively as Kubernetes Secrets. The Operator also provides full secrets lifecycle management, including static and dynamic secrets from the Vault server. The Operator is also capable of managing the rotation of secrets, and notifying applications via rolling updates of a Deployment.

The Vault Secrets Operator supports a variety of authentication methods.

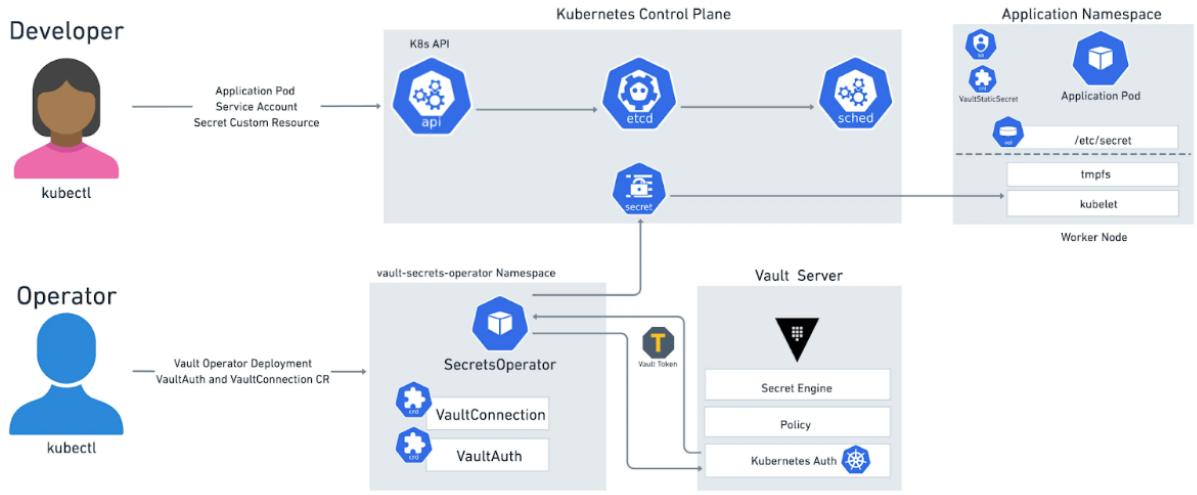


Figure 11: Vault Secrets Operator

11.2 Vault Agent Sidecar Injector

The Vault Agent Sidecar Injector is designed to inject Vault secrets into your applications without requiring the applications to interact with Vault directly. The Agent Sidecar Injector accomplishes this by rendering Vault secrets to shared memory volumes. This approach offers remarkable convenience by allowing containers within a pod to consume secrets without being made Vault-aware. The Vault Agent sidecar Injector functions as a Kubernetes mutating webhook service, intercepting pod events and injecting Vault Agent container(s) into the pod if specific annotations exist on the resource. The Vault Agent Sidecar Injector is a part of the vault-k8s project and can be installed with the Vault Helm chart.

The Vault Agent Sidecar injector supports all authentication methods available to Vault Agent.

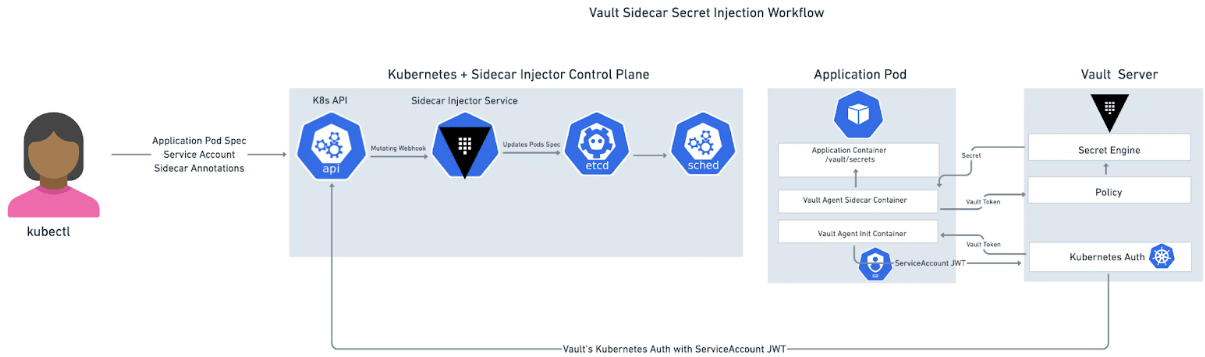


Figure 12: Vault Agent Sidecar Injector

11.3 CSI Provider

The Vault CSI Provider provides secrets to applications running within Kubernetes, operating as a daemonset as opposed to a sidecar within each pod. The CSI Provider utilizes standard Kubernetes CSI Secrets Store volumes. Functionally, a user creates a SecretProviderClass object which specifies `vault` as the secret provider and contains the auth details and secrets to retrieve from Vault. When a Pod is created that references that SecretProviderClass, the Vault CSI provider uses that information along with the pod's service account to retrieve and mount secrets into the pod's CSI volume." This is a blocking operation, as secrets must be retrieved and populated during the ContainerCreation phase, so a pod will not start until the secret has been read and written to the volume.

The CSI Provider supports Kubernetes and JWT authentication methods.

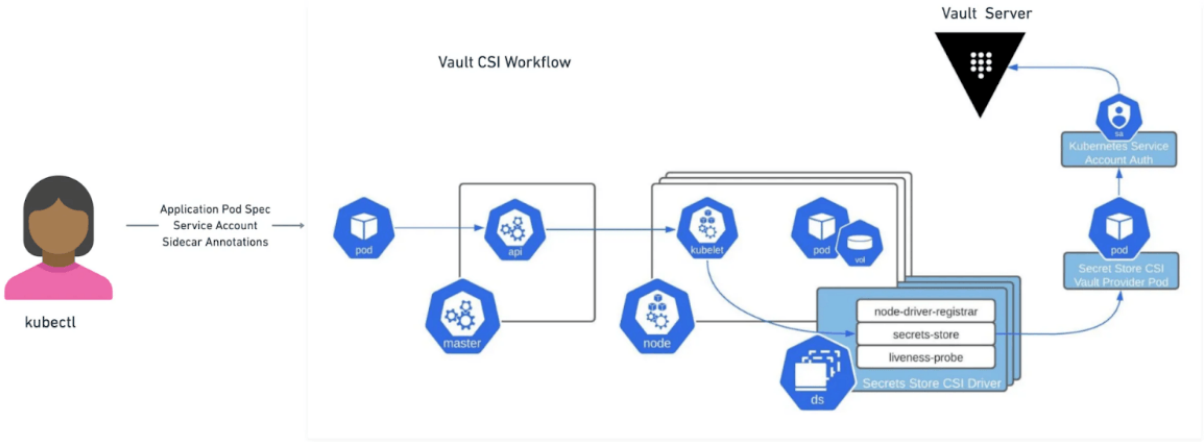


Figure 13: CSI Provider

11.4 Usage Scenarios

11.4.1 Scenario 1

The platform team manages an application on an edge appliance that demands substantial uptime. This application is capable of reading secrets from a CSI volume and can fetch updates without necessitating a pod restart. After evaluating the available options, a sidecar proves to be too resource intensive, and the appliance does not contain multiple Kubernetes nodes so a rolling update is not an option. Ultimately the team settles on the CSI driver for the ability to keep their application up when secrets are updated.

11.4.2 Scenario 2

A platform team wants to leverage multiple Vault instances, and requires the ability to connect to multiple namespaces to retrieve secrets for their applications. They also need to leverage their AWS credentials to authenticate to Vault. The team decides quickly on VSO for its ability to render as native Kubernetes secrets, perform rolling updates, and leverage AWS credentials.

11.5 Additional References

- Kubernetes Vault integration via Sidecar Agent Injector vs. Vault Secrets Operator vs. CSI provider
- VSO Announcement Blog
- Kubernetes documentation about Rolling Updates
- Sidecars and DaemonSets: Battle of containerization patterns

12 Changelog

This page records notable changes to the Vault User Guide.

12.1 2026-08

- Restructured legacy Vault HVD content into the HVD 2.0 Vault User Guide as part of the Installation / Administration / User guide migration.

12.2 2026-07

- Added the RHEL local account password rotation use case, covering the OS secrets engine as a governance pattern for long-lived local accounts.

12.3 2026-06

- Add production hardening guide to prerequisites

12.4 2026-01

- Fixed heading formatting in tabbed content for PDF generation
- Update meta descriptions
- Updated credits and contributors

12.5 2025-12

- Updated credits and contributors

12.6 2025-07

- Added observability and monitoring guidance

12.7 2025-06

- Fixed broken links and added redirects
- Standardized operating guide naming conventions

12.8 2025-05

- Removed document structure section
- Fixed typos and documentation errors
- Fix numbered list rendering

12.9 2025-04

- Updated observability guidance
- Referenced patterns from other HVD documentation

12.10 2024-02

- GA release

13 Credits

Thank you to everyone who contributed to this validated design guidance. This cross-functional team, representing many departments within HashiCorp, authored, edited, reviewed, and iterated

this content to provide prescription and recommendations for using HashiCorp software in enterprise scenarios.

- Adam Cavaliere
- Allison Broski
- Andrew Thielen
- Anna Chen
- Finn Dannin
- Hari Sankaran
- Jan Repnak
- John Duke
- Justin Clayton
- Kalen Arndt
- Mara Hammond
- Mark Lewis
- Mauricio Afiuni
- Nalini Dayma
- Neil Dahlke
- Nick Wong
- Prakash Manglanathan
- Rowan Smith
- Sara Asano
- Sean Doyle
- Theodore Marx
- Troy Fluegge